ROS ESC Tutorial

Author: Nicholas Calkins

Instructor: Dr. Zahra Nili Ahmadabadi

Contents

Abstract

This documentation is a tutorial for how to use the ROS ESC package and other relevant work on the DSIM lab gitlab. In this tutorial, I will be walking through the basic parts of the ROS ESC package, using the Turtlebot3 rotating sensor vehicle as an example. The goal of this tutorial is to show how extremum seeking control designs can be implemented in both Gazebo simulations and real life experiments.

I. Basic ROS Tutorial

I recommend watching this video series, ROS2 Tutorials by the channel Robotics Back-End, for a basic tutorial on ROS2 Humble. We use the ROS2 Humble distribution for all ROS packages on the DSIM lab gitlab.

This tutorial covers the basic ROS2 Humble installation, creation of a ROS workspace, building nodes, publishers, subscribers, and much more. I highly recommend working through the examples presented in this series. A lot of the concepts seen here (particularly with nodes, publishers, and subscribers) are carried over into my work on these ROS packages.

II. Required Items

2.1 Python Libraries

The following Python libraries are needed for the ROS ESC package.

- The numpy library, install via: "pip install numpy"
- The matplotlib library, install via: "pip install matplotlib"
- The sympy library, install via: "pip install sympy"

The following additional Python libraries are needed for the implementation of the ROS ESC package on the Turtlebot3 robotic vehicle.

- The daqhats library for using the MCC172 data acquisition board with a microphone, this can be installed using the install guide here.
- The PySerial library for enabling serial communication between different boards on the vehicle, this can be installed using the installation guide.
- The socket library for communicating with the DSIM lab vicon system, more information is found at the guide here.
- The pigpio library for powering the servo motor and communicating with it's built in encoder via raspberry pi GPIO pins, this can be installed with the installation guide.

2.2 ROS and Gazebo Items

The following items are needed for certain ROS and Gazebo functionality.

- Gazebo and RVIZ simulators, can be installed with the following commands.

```
cd ~
sudo apt install ros-humble-gazebo-ros-pkgs
sudo apt-get install ros-humble-rviz
```

- Gazebo ROS2 Control, can be downloaded by following the instructions in this guide.

2.3 DSIM Lab Gitlab Packages

Note the following packages all originate from the DSIM lab gitlab. To install packages via git, the user must install git by following this guide here, or by using the following commands for installation on Linux.

```
sudo apt-get update

sudo apt-get install git-all
```

Note that some of the following commands assume the user is downloading this package into a ROS workspace named "ros2_ws" under their home directory denoted by the tilde character. If the user is downloading this package elsewhere, please update the terminal commands shown here accordingly.

2.3.1 The ROS ESC Package

The ROS ESC package is used for implementing custom extremum seeking controller designs using ROS. The ROS ESC package from the DSIM Lab Gitlab, can be downloaded and installed via git using the following commands.

```
user@machine:~$ cd ~/ros2_ws/src

user@machine:~$ git clone https://gitlab.com/dsim-lab/ros-packages/ros-esc.git

user@machine:~$ cd ros-esc

user@machine:~$ ~/ros2_ws/src/ros-esc$ pip install -e .
```

2.3.2 The ROS ESC Interfaces Package

The ROS ESC Interfaces package contains custom messages and services used for communication by nodes contained in the ROS ESC package. The ROS ESC Interfaces package from the DSIM Lab Gitlab, can be downloaded and installed via git using the following commands.

```
user@machine:~$ cd ~/ros2_ws/src

user@machine:~$ git clone https://gitlab.com/dsim-lab/ros-packages/ros_esc_interfaces.git
```

2.3.3 The Turtlebot3 Rotating Sensor Gazebo Package

The Turtlebot3 Rotating Sensor Gazebo package contains the URDF file that describes a Turtlebot3 Burger model with a mounted rotating sensor frame. This package also provides several launch files to enable a user to utilize this URDF model in simulation. The Turtlebot3 Rotating Sensor Gazebo package can be downloaded and installed via git using the following commands.

```
user@machine:~$ cd ~/ros2_ws/src

user@machine:~$ git clone https://gitlab.com/dsim-lab/robots/turtlebot-3

                /turtlebot3_rotating_sensor.git
```

2.3.4 The Turtlebot3 Vehicle Nodes Package

The Turtlebot3 Vehicle Nodes package contains additional nodes the facilitate communication with onboard sensors and motors. This package also provides several launch files to enable a user to utilize the ROS ESC package to conduct real life experiments. The Turtlebot3 Vehicle Nodes package can be downloaded and installed via git using the following commands.

```
user@machine:~$ cd ~/ros2_ws/src

user@machine:~$ git clone https://gitlab.com/dsim-lab/robots/turtlebot-3
```

```
/turtlebot3_vehicle_nodes.git
```

This package should also be installed in the appropriate ROS2 workspace on the vehicle's internal computer system.

2.3.5 The Extremum Seeking Package

The ROS ESC package makes use of the Extremum Seeking package on the DSIM lab gitlab. ROS ESC needs to import basic filter and parameter ODE objects built in the extremum seeking package for use in creating custom filter designs. Note that this package should not be included in the user's ROS workspace directory, rather it can be stored directly under the home directory. The extremum-seeking package from the DSIM Lab Gitlab, can be downloaded and installed via git using the following commands.

```
user@machine:~$ cd ~

user@machine:~$ git clone https://gitlab.com/dsim-lab/extremum-seeking/extremum-seeking.git

user@machine:~$ cd extremum-seeking

user@machine:~$ ~/extremum-seeking$ pip install -e .
```

2.3.6 Building the ROS Workspace

Once installation of these packages is complete, the user should build the ROS workspace with the following commands.

```
user@machine:~$ cd ~/ros2_ws

user@machine:~$ colcon build
```

They can also build select packages using the following commands.

```
user@machine:~$ cd ~/ros2_ws

user@machine:~$ colcon build --packages-select ros_esc ros_esc_interfaces

                turtlebot3_rotating_sensor turtlebot3_vehicle_nodes
```

2.4 Additional Items

Although the ROS ESC package is almost entirely written in python, there are some additional things that it would benefit the user to have.

- A working version of MATLAB is useful for some scripts in ROS ESC that do some 3D plotting
- A working version of Arduino IDE is useful for interacting with some scripts implemented on the Turtlebot3 vehicle. Currently, the Arduino IDE is used to upload a script to work with a photoresistor sensor.

III. Robotic Vehicle Modeling

The ROS ESC package can be used to implement extremum seeking controllers on a variety of robotic vehicles. It is designed to be rather abstract so that the same package can be used across the fleet of vehicles in the DSIM lab. Before using the ROS ESC package, a user needs a good vehicle model to work with in simulation. This section describes my process for creating a model of the Turtlebot3 vehicle with a custom rotating sensor frame. This model can be used to conduct Gazebo simulations with the ROS ESC package. Users trying to create a custom robot model for a new vehicle should be familiar with universal robot description files, or URDF for short. ROS has a helpful tutorial for learning how to create URDF files for a robot, we have copied and saved this on the DSIM lab ONR-ESC drive at this link for your reference.

3.1 Custom Meshes for Better Visuals

In the following sections, we will be conducting simulations in RVIZ and Gazebo with a custom Turtlebot3 model with a rotating sensor. In these RVIZ and Gazebo simulators, you can use custom meshes to improve the visuals in simulation. Using your 3D modeling software of choice, export any custom parts you would like to use in the simulation as a mesh in the .dae file format. For the Turtlebot3 rotating sensor package, we are adding several custom parts to build the rotating sensor and improve the chassis visual. The custom meshes used in the Turtlebot3 rotating sensor package are found at the following location.

```
~/ros2_ws/src/turtlebot3_rotating_sensor/meshes
```

In addition, we create a config file to describe our model to Gazebo, this can be found at the location below.

```
~/ros2_ws/src/turtlebot3_rotating_sensor/models/model.config
```

In order for the robot with custom meshes to appear correctly in simulation, the user must copy and paste certain files within their package into the appropriate ".gazebo/models" directory. This is the location where Gazebo will look for meshes described in the robot's URDF file. Use the following commands to perform this action for the Turtlebot3 rotating sensor package.

```
user@machine:~$ cd ~

user@machine:~$ mkdir ~/.gazebo/models/turtlebot3_rotating_sensor

user@machine:~$ cp ~/ros2_ws/src/turtlebot3_rotating_sensor/models/model.config

                ~/.gazebo/models/turtlebot3_rotating_sensor/

user@machine:~$ mkdir ~/.gazebo/models/turtlebot3_rotating_sensor/meshes

user@machine:~$ cp -r ~/ros2_ws/src/turtlebot3_rotating_sensor/meshes

                ~/.gazebo/models/turtlebot3_rotating_sensor/meshes
```

3.2 URDF Model

In this section, we will be taking a look at the URDF model for the Turtlebot3 with the mounted rotating sensor. This can be found in the Turtlebot3 rotating sensor Gazebo package at the following location.

```
~/ros2_ws/src/turtlebot3_rotating_sensor/urdf/turtlebot3_rotating_sensor.urdf
```

3.2.1 Joints and Links

A robot's URDF file models the vehicle as a bunch of interconnected links and joints. Think of a link as a component of a robot, i.e. an arm piece, a wheel, a vehicle chassis etc. A joint will connect one link to another, and describe how these two links will interact with one another. When creating a URDF file for a custom vehicle, you need to start with a link that represents the vehicle's footprint in the environment. For most cases, think of this footprint as the vehicle's center of mass. This is the link used to calculate the vehicle's (x,y,z) position in the global coordinate system within a simulation. All additional links need to be built up on top of this link.

In the following code block, we see our first examples or links and joints. In "base_joint" we use a fixed joint to connect the child link "base_link" to the parent link "base_footprint". By specifying the type of joint used, we have told the simulator that the links are fixed with respect to one another. Under "base_link" we can see three sub items that we need to configure for every new link we add: visual, collision, and inertial. The user should configure new links with appropriate values to describe the vehicle as best as they can (note any angles specified must be in radians). The model does not have to be perfect, but it needs to be accurate enough to get decent simulation results.

```
<!-- Note the base footprint is used to calculate the
turtlebot's position in global (x,y) coordinates. -->

<link name="base_footprint"/>

<joint name="base_joint" type="fixed">
  <parent link="base_footprint"/>
  <child link="base_link"/>
  <origin xyz="0.0 0.0 0.010" rpy="0 0 0"/>
</joint>

<link name="base_link">
  <visual>
    <origin xyz="0 0 0.0" rpy="0 0 3.14"/>
    <geometry>
      <mesh filename="package://turtlebot3_rotating_sensor/meshes/turtlebot_vehicle_model.dae" scale="1 1 1"/
          >
    </geometry>
  </visual>

  <collision>
    <origin xyz="0 0 0.165" rpy="0 0 0"/>
    <geometry>
      <box size="0.140 0.140 0.33"/>
    </geometry>
  </collision>

  <!-- Note that the following inertial calculation is for a simple
  box like shape. The mass value is the weight of the turtlebot, and
  the dimensions of the collision box were used to arrive at the
  following results. Note that this simple box inertia calculation
  assumes that the mass is equally distributed throughout the shape
  of the box, for the real life vehicle, this is not the case. On
  the vehicle, most of the mass is concentrated near the bottom of
  the vehicle where the driving servos, battery, and various boards
  and wires are housed. By default, the inertial origin is right on
  the base link, near the bottom of the model. We do not shift the
  origin of the inertial model because we would like to keep the
  center of mass for this vehicle near the bottom of the robot
  to more closely match the real life vehicle. -->

  <inertial>
    <origin xyz="0 0 0" rpy="0 0 0.13"/>
    <mass value="1.793"/>
    <inertia ixx="1.92e-02" ixy="0" ixz="0" iyy="1.92e-02" iyz="0" izz="5.857e-03" />
  </inertial>
</link>
```

For users creating a URDF model for a new vehicle, I recommend first using RVIZ to visualize and help debug your URDF file. RVIZ is helpful since it provides a lot of tools to help one visualize the link frames and test the joint connections to ensure the model behaves appropriately. Use RVIZ mostly to check if the links and joints are in the right spots, and that any joints function correctly. We will configure collisions and inertial properties later in Gazebo. To start with RVIZ, I would first recommend creating a robot description launch file similar to the one seen in this package at the following location.

`~/ros2_ws/src/turtlebot3_rotating_sensor/launch/robot_description.launch.py`

Within this file, you see an example of how the vehicle's URDF file is published using the robot state publisher node. This means that you have published your vehicle model with ROS, and now the RVIZ and Gazebo simulators in addition to other ROS nodes can interact with that model. Once you have a robot description launch file for your custom robot, you can launch an RVIZ simulation with commands similar to the following. Note we write these commands for the Turtlebot3 rotating sensor example and within multiple terminals.

Terminal 1:

```
user@machine:~$ cd ~/ros2_ws
user@machine:~$ source install/setup.bash
user@machine:~$ rviz2
```

Terminal 2:

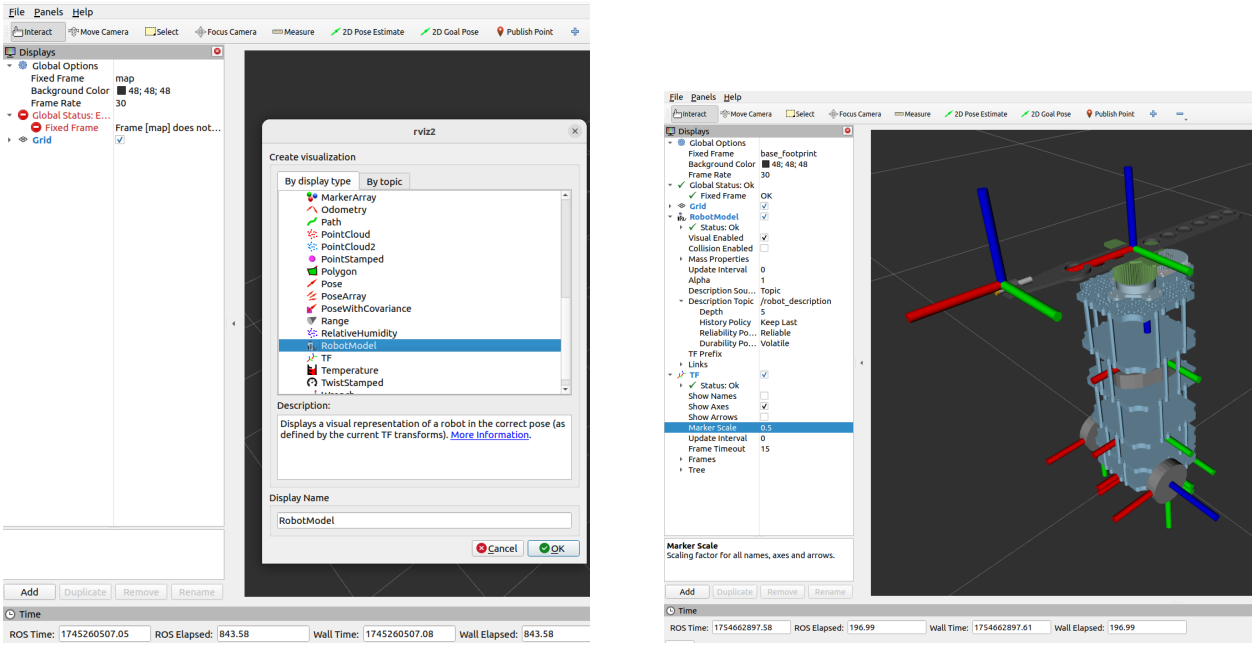
```
user@machine:~$ cd ~/ros2_ws
user@machine:~$ colcon build --packages-select turtlebot3_rotating_sensor
user@machine:~$ source install/setup.bash
user@machine:~$ ros2 launch turtlebot3_rotating_sensor robot_description.launch.py
```

Terminal 3:

```
user@machine:~$ cd ~/ros2_ws
user@machine:~$ source install/setup.bash
user@machine:~$ ros2 run joint_state_publisher_gui joint_state_publisher_gui
```

Now that we have launched everything we need for RVIZ, let us configure the simulation to show the vehicle model. Click the add button on the bottom left of the RVIZ window, in the pop up window, add both the RobotModel and TF items to the display, this can be seen in figure 1a. Once these items are in the display, we need to configure them. Under Global Options on the top left of the side bar, set the Fixed Frame to be "base_footprint" (or your appropriately named link). Then, open up the RobotModel item with the arrow, and set the Description Topic to be "/robot_description". This means that the URDF model we published with the robot description publish python script is now being read by the RVIZ simulation! Finally you can open the TF item and configure it to show the TF frames for all the links similar to what is shown in figure 1b. To test the joints, you can use the joint state publisher gui window and move the sliding bars to see if the joints move appropriately. For the Turtlebot3 model, we have three joints that can move, these are visualized in figure 2. Note that you can save this RVIZ configuration for use the next time you open an RVIZ simulation by going to File on the top left, and selecting the Save Config As option. For the Turtlebot3 rotating sensor package, I have a preset RVIZ configuration saved at the following location.

```
~/ros2_ws/src/turtlebot3_rotating_sensor/rviz_config/urdf_vis.rviz
```



(a) (b)

Figure 1: This shows you where to (a) add the RobotModel and TF items to the RVIZ display pannel, and (b) how to configure these two items so that the Turtlebot model shows up in RVIZ.

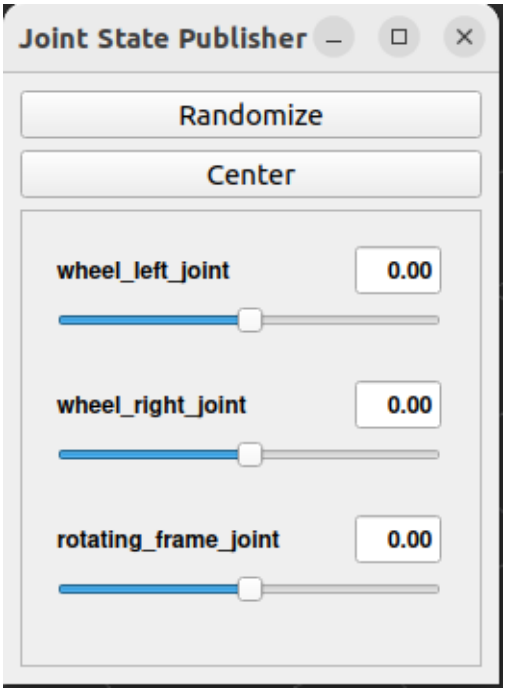
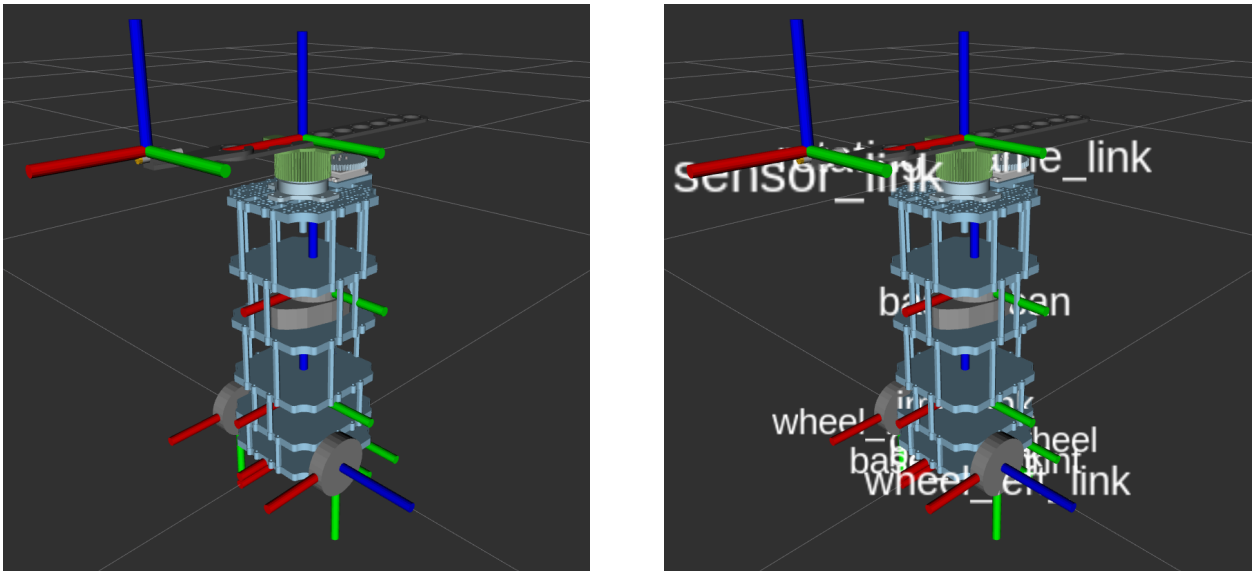


Figure 2: The joint state publisher gui window for the Turtlebot3 rotating sensor model.



(a) (b)

Figure 3: The Turtlebot3 rotating sensor model in a RVIZ simulation with (a) TF frames visualized for every link, and (b) TF frames displayed with the names setting enabled.

3.2.2 Collision and Inertial Model

Once you have worked out the link and joint connections in RVIZ, it is time to move onto a Gazebo simulation to work on the collision and inertial properties of the links. To get started, we need to create a few new launch files for Gazebo specifically. Along with the robot description launch script we made for RVIZ, we will also need a file to launch an empty gazebo world, an example is found at this location.

```
~/ros2_ws/src/turtlebot3_rotating_sensor/launch/empty_world.launch.py
```

In addition, we begin writing a main gazebo launch file for launching our ROS nodes, an example is found at this location.

```
~/ros2_ws/src/turtlebot3_rotating_sensor/launch/gazebo.launch.xml
```

There are a lot of things in this Gazebo launch file which will be covered later in this tutorial. For now we focus on two blocks of code which allow us to spawn the vehicle model in Gazebo. We set the default initial position, initial orientation, and the robot name with the following lines.

```
<!-- Set the default entity name -->
<arg name="entity_name" default="turtlebot3" />

<!-- Set the default initial position -->
<arg name="init_x_position" default="0" />
<arg name="init_y_position" default="0" />
<arg name="init_z_position" default="0" />

<!-- Set the default initial orientation in radians -->
<arg name="init_roll_angle" default="0" />
<arg name="init_pitch_angle" default="0" />
<arg name="init_yaw_angle" default="0" />
```

We then create and configure a ROS node which spawns the robot in an active Gazebo simulation. Note that just like in RVIZ, the robot model is read from the robot description topic to be modeled correctly in the simulator.

```
    <!-- Read robot_description topic and spawn in gazebo -->
    <node
        pkg="gazebo_ros"
        exec="spawn_entity.py"
        name="spawn_entity"
        args="
            -topic=/robot_description
            -entity $(var entity_name)
            -x $(var init_x_position)
            -y $(var init_y_position)
            -z $(var init_z_position)
            -R $(var init_roll_angle)
            -P $(var init_pitch_angle)
            -Y $(var init_yaw_angle)
        "
        output="screen"
    />
```

This XML file is responsible for running several sub launch files in succession to begin a simulation. In the following example, we can see that this XML file runs three different python launch files. First, the Gazebo empty world launch script we mentioned earlier is run, this starts Gazebo with an empty world. Next, the vehicle's URDF file is published to the robot description topic, it is made available for ROS to work with. Finally, a script launches some Gazebo ROS controllers which will be discussed later, this line can be commented out of the launch file for now. After launching these files, it then launches any remaining nodes or executables.

```
<!-- Launch the Gazebo world -->
    <include file="$(find-pkg-share turtlebot3_rotating_sensor)/launch/empty_world.launch.py"/>

    <!-- Publish URDF file in robot_description topic -->
    <include file="$(find-pkg-share turtlebot3_rotating_sensor)/launch/robot_description.launch.py"/>

    <!-- Load the controllers -->
    <include file="$(find-pkg-share turtlebot3_rotating_sensor)/launch/control.launch.py"/>
```

With these launch files setup, we can now begin a Gazebo simulation with the following commands.

```
user@machine:~$ cd ~/ros2_ws

user@machine:~$ colcon build --packages-select turtlebot3_rotating_sensor

                ros_esc ros_esc_interfaces

user@machine:~$ source install/setup.bash

user@machine:~$ ros2 launch turtlebot3_rotating_sensor gazebo.launch.xml
```

With an active Gazebo simulation, we can take a look at several different views of the model by using the View tab on the top left. In figure 4b, we can see the collision model of the vehicle. Collisions are the elements that simulate the physical properties of rigid bodies. Rigid bodies do not change their shape no matter the force applied to them. For the case of this Turtlebot, the collision model for each link is about the same size as the visual model seen in figure 4a. When designing a collision model for a new design, I recommend sticking to simple geometries such as boxes, cylinders, and spheres. In figure 5a, we see the inertia model of the vehicle. The inertia property simulates the mass and the distribution of the mass of each link. In order to get this property right for a new vehicle design, you would like to have decent mass estimates of each link in the URDF file. With that knowledge, one can then use the following equations to

calculate the inertia for these basic shapes.

Box Inertia:

Where m is mass in kg of the link, x, y, and z are the side lengths in meters,

$$
\begin{aligned}
I_{xx} &= m\frac{y^2+z^2}{12} \\
I_{xy} &= 0 \\
I_{xz} &= 0 \\
I_{yy} &= m\frac{x^2+z^2}{12} \\
I_{yz} &= 0 \\
I_{zz} &= m\frac{x^2+y^2}{12}
\end{aligned}
$$

Cylinder Inertia:

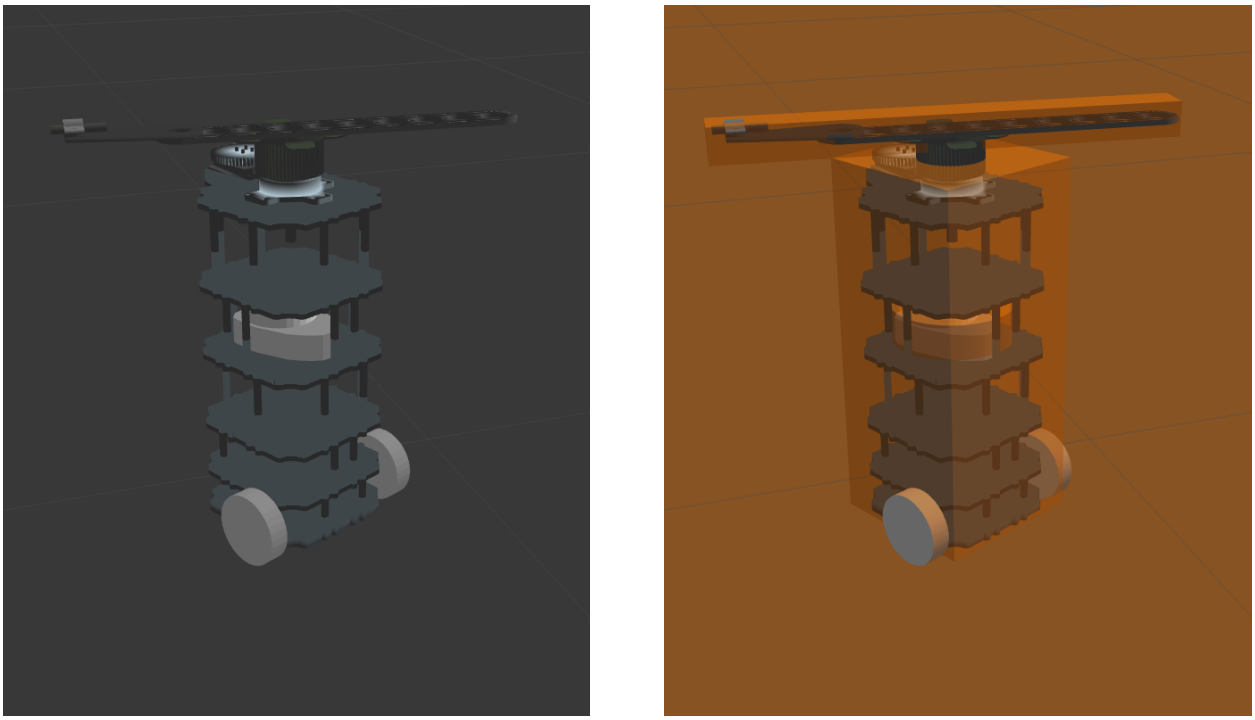
Where m is mass in kg of the link, r is radius of the cylinder in meters, and h is the height of the cylinder in meters (note the face of the cylinder is in the x, y plane),

$$
\begin{aligned}
I_{xx} &= m\frac{3r^2+h^2}{12} \\
I_{xy} &= 0 \\
I_{xz} &= 0 \\
I_{yy} &= m\frac{3r^2+h^2}{12} \\
I_{yz} &= 0 \\
I_{zz} &= m\frac{r^2}{2}
\end{aligned}
$$

Sphere Inertia:

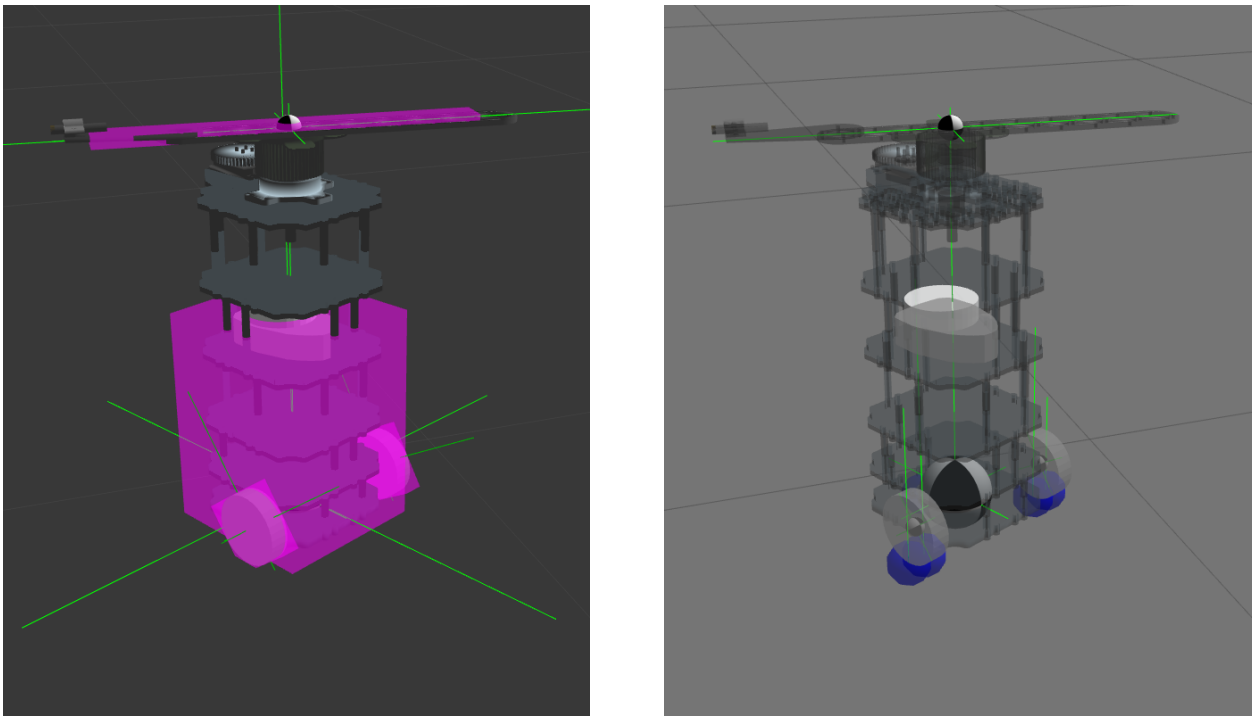
Where m is mass in kg of the link, r is radius of the sphere in meters,

$$
\begin{aligned}
I_{xx} &= m\frac{2r^2}{5} \\
I_{xy} &= 0 \\
I_{xz} &= 0 \\
I_{yy} &= m\frac{2r^2}{5} \\
I_{yz} &= 0 \\
I_{zz} &= m\frac{2r^2}{5}
\end{aligned}
$$



(a) (b)

Figure 4: The Turtlebot3 rotating sensor model in a Gazebo simulation with (a) the visual model with custom meshes, and (b) the collision model of all the links.



(a) (b)

Figure 5: The Turtlebot3 rotating sensor model in a Gazebo simulation with (a) the inertia model with pink inertia frames, and (b) the vehicle contact points with the ground.

3.3 Plugins

Near the end of the URDF file, one should include any plugins they would like to use with their model. For the Turtlebot3 vehicle, the first of these is the differential drive plugin seen in the URDF portion below. This plugin tells Gazebo that this vehicle should adhere to differential drive kinematics when we command it to move in simulation. For other vehicle designs with different kinematic constraints, they will need to search and find an appropriate plugin to get the movement properties they require.

```
      <!-- Differential Drive -->
<gazebo>
  <plugin name="turtlebot3_diff_drive" filename="libgazebo_ros_diff_drive.so">

    <ros>
      <!-- <namespace>/tb3</namespace> -->
    </ros>

    <update_rate>30</update_rate>

    <!-- wheels -->
    <left_joint>wheel_left_joint</left_joint>
    <right_joint>wheel_right_joint</right_joint>

    <!-- kinematics -->
    <wheel_separation>0.160</wheel_separation>
    <wheel_diameter>0.066</wheel_diameter>

    <!-- limits -->
    <max_wheel_torque>20</max_wheel_torque>
    <max_wheel_acceleration>1.0</max_wheel_acceleration>

    <command_topic>cmd_vel</command_topic>
```

```
    <!-- output -->
    <publish_odom>true</publish_odom>
    <publish_odom_tf>true</publish_odom_tf>
    <publish_wheel_tf>false</publish_wheel_tf>

    <odometry_topic>odom</odometry_topic>
    <odometry_frame>odom</odometry_frame>
    <robot_base_frame>base_footprint</robot_base_frame>

  </plugin>
</gazebo>
```

This next plugin is the joint state publisher. This will constantly publish the state of the joints using a ROS node. Note that we need to tell this joint publisher to broadcast the state of all of the rotating frame joints on the vehicle model. This information will be used to get the angular position of these joints later on.

```
      <!-- Joint Publisher -->
<gazebo>
  <plugin name="turtlebot3_joint_state" filename="libgazebo_ros_joint_state_publisher.so">
    <ros>
      <!-- <namespace>/tb3</namespace> -->
      <remapping>~/out:=joint_states</remapping>
    </ros>
    <update_rate>30</update_rate>

    <!-- Publish the rotating frame joints in order here.
    These joints will appead in the joint state message
    denoted by their joint names in the "name" category
    in the order they are listed below. -->
    <joint_name>rotating_frame_joint</joint_name>
  </plugin>
</gazebo>
```

3.4 Gazebo ROS2 Control

We make use of the Gazebo ROS2 Control to set and control the angular velocity of our rotating sensor frames. To do this we need to specify some initial configurations in the URDF portion below. For the rotating sensor frame joint, we are commanding the joint's angular velocity property, and we set the state interface to be the joint's angular position. We can set min and max angular velocity constraints, as well as setting initial values for the angular position and angular velocity. Note that all of these values are in units of radians and radians per second. For vehicle models that have multiple rotating sensor frames, they can specify additional joints to control within a code block like this by copying the format for the rotating frame joint seen here. Please be sure to give these joints distinctive names.

```
<!-- Rotating Frame Velocity Controller -->
<ros2_control name="GazeboSystem" type="system">
    <hardware>
        <plugin>gazebo_ros2_control/GazeboSystem</plugin>
    </hardware>

    <joint name="rotating_frame_joint">
        <command_interface name="velocity">
            <param name="min">0</param>
            <param name="max">5</param>
        </command_interface>
```

```
        <state_interface name="position">

          <!-- Use this parameter here to set the
          intial angular position of the frame -->
          <param name="initial_value">-1.57</param>

        </state_interface>
        <state_interface name="velocity">
          <param name="initial_value">0.0</param>
        </state_interface>
    </joint>
  </ros2_control>

  <gazebo>
      <plugin name="gazebo_ros2_control" filename="libgazebo_ros2_control.so">
          <!-- This is the yaml file that describes the controllers we want to use -->
          <parameters>$(find turtlebot3_rotating_sensor)/controller_config/joint_controller.yaml</parameters>
          <robot_param_node>/robot_state_publisher_node</robot_param_node>
      </plugin>
  </gazebo>
```

In addition to the above URDF portion, to fully set up Gazebo ROS2 Control, we need to create another controller configuration file. An example Gazebo ROS2 controller config file for the Turtlebot3 can be seen at the following location.

`~/ros2_ws/src/turtlebot3_rotating_sensor/controller_config/joint_controller.yaml`

Looking at this control config file, we see under controller manager, we list both the Gazebo ROS2 controller (named "velocity_controller") and the joint state broadcaster, and set their types. For vehicles with multiple rotating sensor frames, the user should add more controllers (with distinctive names) to be configured under the controller manager the same way as shown below. After completing the controller manager configuration, all the Gazebo ROS2 controllers need some additional information seen under "velocity_controller" below. Here we specify the joint name to be controlled, set the command interface to be the joint's angular velocity, and then add the state interfaces. For vehicles with multiple rotating sensors, one should have a code block like this for each individual Gazebo ROS2 controller.

```
controller_manager:
  ros__parameters:
    update_rate: 100 #Hz
    use_sim_time: true
    velocity_controller:
      type: velocity_controllers/JointGroupVelocityController
    joint_state_broadcaster:
      type: joint_state_broadcaster/JointStateBroadcaster

velocity_controller:
  ros__parameters:
    joints:
      - rotating_frame_joint
    interface_name: velocity
    command_interfaces:
      - velocity
    state_interfaces:
      - position
      - velocity
```

In addition to this config file, we create a python script to launch these velocity controllers and joint state broadcasters. An example controller launch script can be seen at the following location.

`~/ros2_ws/src/turtlebot3_rotating_sensor/launch/control.launch.py`

We add this sub launch file to our Gazebo launch XML file shown below. Our launch sequence for every Gazebo simulation becomes the following

1. Launch the empty gazebo world
2. Publish the URDF file to the robot description topic
3. Launch our velocity controllers and joint state broadcaster
4. Spawn the vehicle in the simulation and launch additional ROS nodes

```
<?xml version='1.0' ?>

<launch>
    <!-- Launch the Gazebo world -->
    <include file="$(find-pkg-share turtlebot3_rotating_sensor)/launch/empty_world.launch.py"/>

    <!-- Publish URDF file in robot_description topic -->
    <include file="$(find-pkg-share turtlebot3_rotating_sensor)/launch/robot_description.launch.py"/>

    <!-- Load the controllers -->
    <include file="$(find-pkg-share turtlebot3_rotating_sensor)/launch/control.launch.py"/>

    <!-- Read robot_description and spawn in gazebo, run the nodes -->
    <include file="$(find-pkg-share turtlebot3_rotating_sensor)/launch/gazebo.launch.py"/>
</launch>
```

The complete Gazebo launch XML file can be found at the following location.

`~/ros2_ws/src/turtlebot3_rotating_sensor/launch/gazebo.launch.xml`

3.5 Troubleshooting

In this section I briefly discuss some issues I was having while making the Turtlebot3 model and the solutions I came up with to resolve them. One issue that arose when first designing the vehicle model was that it had a tendency to slowly drift around with no movement input after spawning in. If we look at the contact view seen in figure 5b, we can see there are blue circles representing the three points of contact (two wheels and one castor wheel) the vehicle has with the ground. When I was having these drifting issues, these contact points were constantly flickering on and off. You do not want to see these flickering contacts between the wheels and the ground because it indicates poor contact detection in the simulation. My workaround for this issue was altering the wheel contact stiffness properties kp, kd, mu1, and mu2. Looking at the wheel link portion of the URDF file, we can see that I am changing these properties under the gazebo reference tag after describing the wheel link. I have see these high enough so that the wheel doesn't sink into the ground, but low enough to not be too rigid, which causes poor contact with the ground. Additionally, under the wheel joint tag, I am adding damping and friction to the joint properties. These changes seemed to resolve the drifting issue.

```
    <joint name="wheel_left_joint" type="continuous">
    <parent link="base_link"/>
    <child link="wheel_left_link"/>
    <origin xyz="0.032 0.08 0.023" rpy="-1.57 0 0"/>
    <axis xyz="0 0 1"/>
    <limit effort="10000" velocity="1000"/>
    <joint_properties damping="1.0" friction="100.0"/>
  </joint>

    <link name="wheel_left_link">
    <visual>
      <origin xyz="0 0 0" rpy="1.57 0 0"/>
      <geometry>
```

```
      <mesh filename="package://turtlebot3_rotating_sensor/meshes/tire.dae" scale="0.001 0.001 0.001"/>
    </geometry>
  </visual>

  <collision>
    <origin xyz="0 0 0" rpy="0 0 0"/>
    <geometry>
      <cylinder length="0.018" radius="0.033"/>
    </geometry>
  </collision>

  <inertial>
    <origin xyz="0 0 0" />
    <mass value="2.8498940e-02" />
    <inertia ixx="1.1175580e-05" ixy="-4.2369783e-11" ixz="-5.9381719e-09" iyy="1.1192413e-05" iyz="
        -1.4400107e-11" izz="2.0712558e-05" />
  </inertial>
</link>

<!-- Wheel contact stiffness kp and kd have been adjusted
These values have been chosen since they are high enough
so that the wheel doesn't sink into the ground, but still
low enough to not be "too rigid" which causes poor contact
detection between the wheel and the ground. Both the left
and right wheel should each have two constant (non-flickering)
points of contact with the ground. Stable contact points
stop the model from drifting laterally in simulation. -->

<gazebo reference="wheel_left_link">
  <!-- Set static contact stiffness, higher is more rigid -->
  <kp>5000.0</kp>
  <!-- Set dynamic contact stiffness, higher is more rigid -->
  <kd>5000.0</kd>
  <!-- Set static friction coefficient -->
  <mu1>100000</mu1>
  <!-- Set dynamic friction coefficient -->
  <mu2>10000</mu2>
</gazebo>
```

The next issue I was running into was with the castor wheel. The URDF format does not have a spherical joint type which would best capture how the castor wheel actually moves underneath the vehicle. Instead I had to model it as a fixed sphere with the URDF portion seen below. When driving the vehicle around, this caster wheel generated friction with the ground, creating poor and unrealistic movement qualities with quick turns and the like. In order to better capture the real life caster wheel, I again used the gazebo reference tag and set the friction coefficients very low. This essentially makes the caster wheel "frictionless". This improved the driving qualities of the vehicle.

```
  <!-- Note the caster wheel is modeled as a "frictionless"
fixed sphere since there is no "ball" joint in URDF files
like there is in the SDF format. The caster wheel will not
roll in simulation, it is just a placeholder to have three
points of contact between the turtlebot and the ground. -->

<joint name="caster_wheel_joint" type="fixed">
  <parent link="base_link"/>
  <child link="caster_wheel"/>
  <origin xyz="-0.049 0 -0.005" rpy="0 0 0"/>
</joint>

<link name="caster_wheel">
  <visual>
    <origin xyz="0 0 0" rpy="0 0 0"/>
    <geometry>
```

```
      <sphere radius="0.005"/>
    </geometry>
  </visual>

  <collision>
    <origin xyz="0 0 0" rpy="0 0 0"/>
    <geometry>
      <sphere radius="0.005"/>
    </geometry>
  </collision>

  <inertial>
    <origin xyz="0 0 0" rpy="0 0 0"/>
    <mass value="0.005" />
    <inertia ixx="7.2e-8" ixy="0.0" ixz="0.0" iyy="7.2e-8" iyz="0.0" izz="7.2e-8" />
  </inertial>
</link>

<!-- Setting the caster wheel friction very low,
i.e. making it almost "frictionless". -->
<gazebo reference="caster_wheel">
  <mu1 value="0.001"/>
  <mu2 value="0.001"/>
</gazebo>
```

The final issue I was dealing with in the URDF file was with the rotating sensor frame. When the rotating frame was spinning atop the vehicle, it was causing the vehicle's trajectory to drift with the spinning frame as well. In real life when the rotating frame spins, the wheels have enough friction with the ground to resist the torque caused by the rotating frame on the vehicle's body. To mimic this in simulation, I have set the joint friction and damping to be very low. Note this may not be an appropriate work around for all vehicle designs, it may be the case that the rotating sensor frame does apply some kind of moment to affect your vehicle's heading in real life. In that case one may need to spend more time to develop a better model of this effect. Later on, we will see how I command this rotating frame to spin atop the vehicle so you can test this for yourself.

```
  <!-- Here, the rotating frame's joint is modeled to be
"frictionless". This is done to limit the drifting of
the turtlebot's trajectory as the frame spins around
atop the turtlebot. In real life, as the frame spins,
the wheels have enough friction to resist the torque
cased by the rotating frame on the vehicle's body. In
simulation we try to mimic this effect by setting this
joint friction and damping to be very low. -->

<joint name="rotating_frame_joint" type="continuous">
  <parent link="servo_link"/>
  <child link="rotating_frame_link"/>
  <origin xyz="0 0 0.0432" rpy="0 0 0"/>
  <axis xyz="0 0 1"/>
  <limit effort="20" velocity="10"/>
  <dynamics damping="0.00001" friction="0.00001"/>
</joint>

<link name="rotating_frame_link">
  <visual>
    <origin xyz="-0.151 0 0" rpy="0 0 1.57"/>
    <geometry>
      <mesh filename="package://turtlebot3_rotating_sensor/meshes/rotating_frame_assembly.dae"/>
    </geometry>
  </visual>

  <collision>
```

```
      <origin xyz="0 0 0.01" rpy="0 0 0"/>
      <geometry>
        <box size="0.325 0.035 0.02"/>
      </geometry>
    </collision>

    <inertial>
      <origin xyz="0 0 0.005" rpy="0.025 0 1.57"/>
      <mass value="5.4e-02" />
      <inertia ixx="4.683e-4" ixy="0" ixz="0" iyy="4.43e-6" iyz="1.111e-5" izz="4.713e-04" />
    </inertial>
  </link>

  <gazebo reference="rotating_frame_link">
    <!-- Set static contact stiffness, higher is more rigid -->
    <kp>10000000000000000000000.0</kp>
    <!-- Set dynamic contact stiffness, higher is more rigid -->
    <kd>10000000000000000000000.0</kd>
    <!-- Set static friction coefficient -->
    <mu1>1000</mu1>
    <!-- Set dynamic friction coefficient -->
    <mu2>100</mu2>
  </gazebo>
```

IV. ROS Nodes

We utilize ROS in order to implement extremum seeking control schemes onto robotic vehicles. ROS is an open source robotic operating system containing development tools, communication infrastructure, and a plethora of preexisting applications for robotic projects. Using ROS architecture, I have been developing software packages for use in ESC design and implementation. The extremum seeking control problem is broken down into sections, where each section is a task managed by a ROS node. By breaking the ESC problem into smaller sections, we can create a modular framework where the ROS nodes can take on different configurations to implement arbitrary ESC scheme designs. This breakdown can be seen in the block diagram in 6. If a user needs to change the functionality of a particular item, it simply involves reconfiguring one node in the system. With this modular design, the goal is to allow for rapid testing and deployment of ESC schemes, first in Gazebo simulation, then on real world robotic vehicles.

This system uses configuration .json files to initialize and tune several of the following ROS nodes. These nodes in the ROS ESC package come associated with objects in the package that can be called to and initialized with these config files. An example of a basic config file can be seen below. This config file is written like a dictionary with keys and parameters. The first two keys that always appear at the top of a config file are the "filepath" and "object_name" keys. The "filepath" key describes the absolute filepath to a python script to reference. Within the reference script there should be an object, whose name is given by the "object_name" key, which the user would like to configure and use with the appropriate ROS node. The "params" key describes a dictionary, whose entries are used to configure the selected object. Example configuration files for each ROS node will be discussed in their respective subsections, this just serves as a brief introduction to the concept here.

```
{
    "dictionary_name":{
        "filepath": "absolute filepath to a python script to reference",
        "object_name": "name of the object from the script to configure",
        "params:{
            "dictionary of parameters to configure the selected object",
        }
    }
}
```

The methodology for using this package is to first test ESC designs in Gazebo experiments, and do most of the parameter tuning in simulation. After verifying a design in Gazebo, these same configuration files can then be uploaded to the robotic vehicle, use the same ROS nodes it used before, and conduct real life experiments.

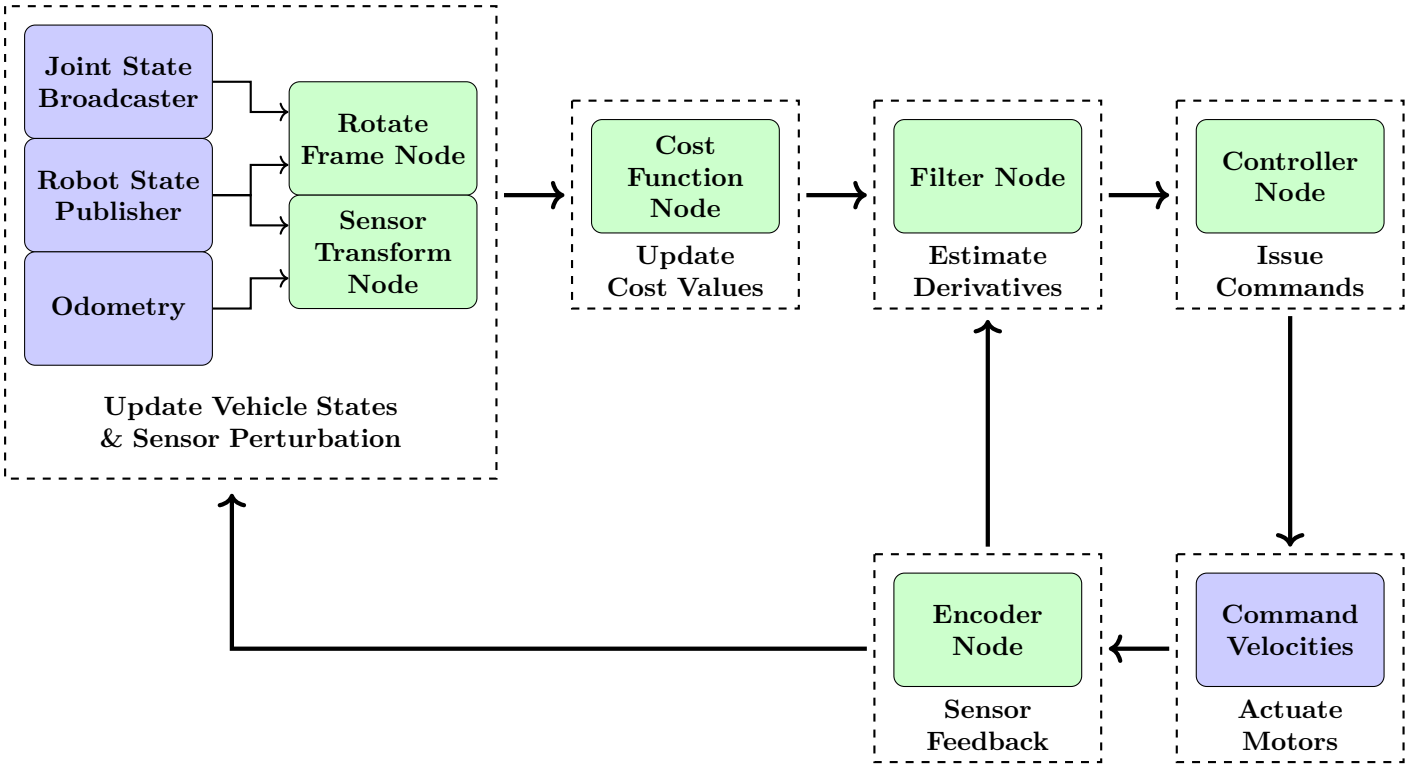


Figure 6: Block diagram of ROS nodes used for a Gazebo simulation. Note any block colored in green comes from the ROS ESC package.

In the following sections I will be describing the various ROS nodes in the ros_esc package. I will detail their purpose, how they can be configured, and describe how they can be launched. The following sections walk through the process of creating the Gazebo launch xml file for the turtlebot3_rotating_sensor package found at the following location. Please feel free to follow along as we describe the launch procedure node by node.

```
~/ros2_ws/src/turtlebot3_rotating_sensor/launch/gazebo.launch.xml
```

4.1 Custom Messages and Services

To facilitate data transmission between the nodes in the ROS ESC package, we have developed several custom message types and service calls which can be found in the ROS ESC interfaces package. The following are brief descriptions of these custom messages and services as of the time of writing this document. For future expansion of the ROS ESC package, any new message types or service calls should be added to the ROS ESC Interfaces package.

- StampedFloat64.msg: A simple wrapper message that allows us to send a standard ROS Float64 message but with a timestamp and header. Float64 message definition can be found here.
- StampedFloat64MultiArray.msg: A simple wrapper message that allows us to send a standard ROS Float64MultiArray message but with a timestamp and header. This is used by most of the nodes in the ROS ESC package to send timestamped data. Float64MultiArray message definition can be found here.

- StampedString.msg: A simple wrapper message that allows us to send a standard ROS String message but with a timestamp and header. String message definition can be found here.
- StampedTransformMultiArray.msg: A message that allows us to send multiple ROS Geometry Transform messages with a timestamp and a header. This is used by the sensor pose node to send multiple sensor transformation matrices to be evaluated by the cost function node. Transform message definition can be found here.
- Timekeeper.msg: This message is used to broadcast timekeeping information for other ROS nodes to reference. This message communicates the starting time of an experiment, and whether timekeeping should be done with Gazebo simulation time, or real time. This message is critical for syncing up the timekeeping between the various ROS nodes that need it primarily the cost function node, the filter node, and the controller node.

4.2 Rotate Sensor Frame Node

The rotate frame node is responsible for controlling the spin of the rotating sensor frame. The user has the freedom to alter the parameters of the rotating sensor frame's spinning by using what I call a spin profile object. This object describes the desired velocity of the rotating sensor frame as a function of the current time, the frame's angular position, and the frame's current spin direction. Additional parameters of a spin profile object must be configured in a rotate frame configuration file.

This node is responsible for publishing timekeeping information which the other ROS nodes will reference. The instant this rotate frame node issues its first velocity command and starts to spin the frames is considered the start time of the experiment. This is then broad casted to all the other ROS nodes for them to reference with a Timekeeper message. This message also comes with a note specifying that we are utilizing simulation time mode over the real time mode. When this message is received by one of the following nodes, it stores this experiment start time and the timekeeping mode internally. When a node needs to generate a timestamp, it first obtains the current time (either real or sim time), then it subtracts the experiment start time from the current time to get the time elapsed. The time elapsed is then used to timestamp every message that gets published.

4.2.1 Launch Arguments

This rotate sensor frame node expects a few input arguments from the user to launch correctly. These are given in the ordered list below.

1. The user must name an input encoder topic, this is used to collect Stamped64MultiArary messages which contain the angular positions of the rotating sensor frames.
2. The user must name an output topic to publish Timekeeper messages to, note the instant the sensor frame starts to rotate is considered the start time of the experiment, this is communicated to other ROS nodes via this message.
3. The user must name a rotate sensor frame configuration file to use, this gives the rotate sensor frame node a configured object to use in an experiment.

These arguments are then used to form the launch command for the rotate frame node seen in 4.2.4.

4.2.2 Rotate Frame Config File

The rotate frame node must be configured with a config file, an example of which is seen below. First note the name "velocity_controller" at top of the config file. This name here must match the name of the one setup in 3.4 since it will be used to setup the output topic to publish velocity commands to. For example, if a velocity controller named "example_velocity_controller" is setup in a file similar to 3.4, the exact same name should appear in a rotate frame config file like this since this node will take that name and publish commands to the topic: "/example_velocity_controller/commands".

The user must supply the absolute file path to a python script to reference, and name an object written in that script to use with this node. The params dictionary is then used to configure that object with items specified by the user. In this example, we have selected the "Constant_Full_Rotation" object from the given filepath. The params dictionary will configure that object with the speed we would like to spin the frame at.

```
{
    "velocity_controller":{
        "filepath": "~/ros2_ws/src/ros_esc/ros_esc/rotate_frame_node/spin_profile_objects/spin_profile_objects.
            py",
        "object_name": "Constant_Full_Rotation",
        "params":{
            "spin_rpm": 20
        }
    }
}
```

More examples of rotate frame config files can be found at the following location.

`~/ros2_ws/src/ros_esc/ros_esc/rotate_frame_node/rotate_frame_config_files`

For users who want to test new config files, they can check to make sure they are parsed correctly using the testing script at the following location.

`~/ros2_ws/src/ros_esc/ros_esc/rotate_frame_node/rotate_frame_config_testing.py`

4.2.3 Spin Profile Objects

This node needs a spin profile object specified by the user to use in an experiment. During an experiment, the rotate sensor node will always give this spin profile object the same three inputs: time, angular position, and spin direction. The node will always expect a velocity output from this spin profile object. However, how one gets from these inputs to a velocity output is completely up to the user to decide when they design a spin profile object.

The above example selects a spin profile object named "Constant_Full_Rotation" from within the ROS ESC package. This object can be found within the python script at the following location.

`~/ros2_ws/src/ros_esc/ros_esc/rotate_frame_node/spin_profile_objects/spin_profile_objects.py`

Looking into this python script, we can see that this spin profile object enables constant full rotation of the frame. The rotate frame node will constantly call the "velocity_output" method of this object during an active experiment. We can see that this velocity output method takes in the three inputs from the rotate frame node we stated before, and returns a velocity output a the bottom. In this case, none of the inputs from the node actually get used since the velocity is held constant regardless of any input parameter. The velocity is kept constant with the parameters specified in the config file.

```
# pylint: disable=too-few-public-methods
# pylint: disable=invalid-name
class Constant_Full_Rotation(SpinProfile):
    """This class enables constant full rotation of the frame."""

    def __init__(self, params):
        """This initializes the spin profile object.

        The params input should be a dictionary with the following keys
        "params":{
            "spin_rpm": float
        }
        """

        # Define the rotation speed in radians per second
        self.speed = params["spin_rpm"]*(2*np.pi/60)

    def velocity_output(self, time, angular_position, spin_direction):
```

```
        """This operates on the input arguments and produces the velocity output.

        time (float): current time
            (either simulation or real time)

        angular_position (float): current angular position of the rotating frame

        spin_direction (boolean or None): current spin direction of the rotating frame
            (True for ccw rotation, False for cw rotation, None if uninitialized)
        """

        # No dependence on any input variable
        output = self.speed

        return output
```

More examples of spin profile objects can be found at the following location.

`~/ros2_ws/src/ros_esc/ros_esc/rotate_frame_node/spin_profile_objects`

4.2.4 Adding Node to Launch File

Before we add the launch arguments for this node specifically, we first add default arguments to launch and spawn the robot.

```
    <!-- Set the default entity name -->
    <arg name="entity_name" default="turtlebot3" />

    <!-- Set the default initial position -->
    <arg name="init_x_position" default="0" />
    <arg name="init_y_position" default="0" />
    <arg name="init_z_position" default="0" />

    <!-- Set the default initial orientation in radians -->
    <arg name="init_roll_angle" default="0" />
    <arg name="init_pitch_angle" default="0" />
    <arg name="init_yaw_angle" default="0" />
```

We publish the robot described in the URDF file with the following commands.

```
    <!-- Launch the Gazebo world -->
    <include file="$(find-pkg-share turtlebot3_rotating_sensor)/launch/empty_world.launch.py"/>

    <!-- Publish URDF file in robot_description topic -->
    <include file="$(find-pkg-share turtlebot3_rotating_sensor)/launch/robot_description.launch.py"/>

    <!-- Load the controllers -->
    <include file="$(find-pkg-share turtlebot3_rotating_sensor)/launch/control.launch.py"/>
```

We launch a node to spawn the robot with the following command.

```
    <!-- Read robot_description topic and spawn in gazebo -->
    <node
        pkg="gazebo_ros"
        exec="spawn_entity.py"
        name="spawn_entity"
        args="
            -topic=/robot_description
            -entity $(var entity_name)
            -x $(var init_x_position)
            -y $(var init_y_position)
            -z $(var init_z_position)
```

```
            -R $(var init_roll_angle)
            -P $(var init_pitch_angle)
            -Y $(var init_yaw_angle)
        "
        output="screen"
    />
```

We now add the following items to the main Gazebo launch file. First, we initialize defaults for the variables necessary to launch this node

```
    <!-- Set the default rotate frame configuration filepath -->
    <arg name="rotate_frame_config_filepath" default="~/ros2_ws/src/ros_esc/ros_esc/rotate_frame_node/
         rotate_frame_config_files/turtlebot_vehicle/full_rotation.json" />
```

We then create a command to launch the rotate sensor frame node with the arguments given in 4.2.1. Note we setup the ROS topic publishing and subscribing with these launch arguments.

```
    <!-- Launch the rotate frame node -->
    <executable cmd="
        ros2 run ros_esc rotate_frame_node
        $(var entity_name)/encoder_chatter
        $(var entity_name)/timekeeper_chatter
        $(var rotate_frame_config_filepath)
    "
    />
```

4.3 Encoder Node

This node is used to obtain the angular positions of the spinning sensor frames. Upon execution, this node publishes the angular positions of the sensor frames in units of radians. This node mimics the information one would get from absolute rotary encoders placed on these rotating frames on a real life vehicle. In simulation, this node will use data from the joint state publisher we already configured in the URDF file in 3.3. Please ensure that all rotating frame joints are specified in this portion of the URDF file.

4.3.1 Launch Arguments

This encoder node expects a few input arguments from the user to launch correctly. These are given in the ordered list below.

1. The user must name an input joint states topic, this is used to collect JointState messages which contain the angular positions of the rotating sensor frames.
2. The user must name an input timekeeper topic, this is used to collect Timekeeper messages which contain timekeeping information, this is used to create timestamps for the output data.
3. The user must name an output topic to publish StampedFloat64MultiArray messages to, these messages contain an array of the angular positions of the rotating sensor frames.
4. After the "–joint_names" keyword, the user must list all the names of the joints they configured in the URDF file seen in 3.3.

These arguments are then used to form the launch command for the encoder node seen in 4.3.2.

4.3.2 Adding Node to Launch File

We now add the following items to the main Gazebo launch file. First, we initialize defaults for the variables necessary to launch this node

```
<!-- Set the joint name of the rotating sensor frame joint from the URDF file -->
<arg name="joint_name_1" default="rotating_frame_joint" />
```

We then create a command to launch the rotate sensor frame node with the arguments given in 4.3.1. Note we setup the ROS topic publishing and subscribing with these launch arguments.

```
<!-- Launch the encoder node -->
<executable cmd="
    ros2 run ros_esc encoder_node
    /joint_states
    $(var entity_name)/timekeeper_chatter
    $(var entity_name)/encoder_chatter
    --joint_names
    $(var joint_name_1)
"
/>
```

4.4 Visualize Active ROS Topics

With the encoder and rotate frame node setup, we can run a Gazebo simulation to see them in action. To start the simulation with the Gazebo launch file we're building up, use the following commands shown below.

```
user@machine:~$ cd ~/ros2_ws

user@machine:~$ colcon build --packages-select ros_esc ros_esc_interfaces

                turtlebot3_rotating_sensor

user@machine:~$ source install/setup.bash

user@machine:~$ ros2 launch turtlebot3_rotating_sensor gazebo.launch.xml
```

When the simulation starts we should see the vehicle spawned in and the sensor frame spinning in a circle. This means that our encoder and rotate frame nodes are operating correctly! With ROS communication, we can actually see what these nodes are publishing in a separate terminal. Looking at figure 7, we can use the command "ros2 topic list" to show all the active ROS topics in our current simulation. We note that the topics "/turtlebot_1/encoder_sim_chatter", "/turtlebot_1/timekeeper_chatter" come from launching the encoder and rotate sensor frame nodes, while the topics "/velocity_controller/commands", and "/velocity_controller/transistion_event" come from initializing the Gazebo ROS2 controllers in section 3.4. In figure 8, we can view the datastream the encoder node is publishing using the command "ros2 topic echo /turtlebot_1/encoder_sim_chatter". We see that a StampedFloat64MultiArray message contains a header, timestamp (with simulation time in this case), and data. Since this vehicle has only one rotating sensor, this data array contains only one value in radians. Note that this value of approximately 214 radians when divided by 2π tells us that this frame completed a full counterclockwise rotation approximately 34 times when I took this screen shot!

Looking at figure 9 we can check if our Gazebo ROS2 Controllers setup in section 3.4 are active with the command "ros2 control list_controllers". In figure 10, we can then see the velocity commands that the rotate sensor frame node is sending to this active controller with the command "ros2 topic echo /velocity_controller/commands". In the rotate sensor frame config file in the previous section, we specified that we want our frame to rotate at a constant speed of 20 RPM. We can see here that the node is publishing that same speed, merely converted to radians per second.

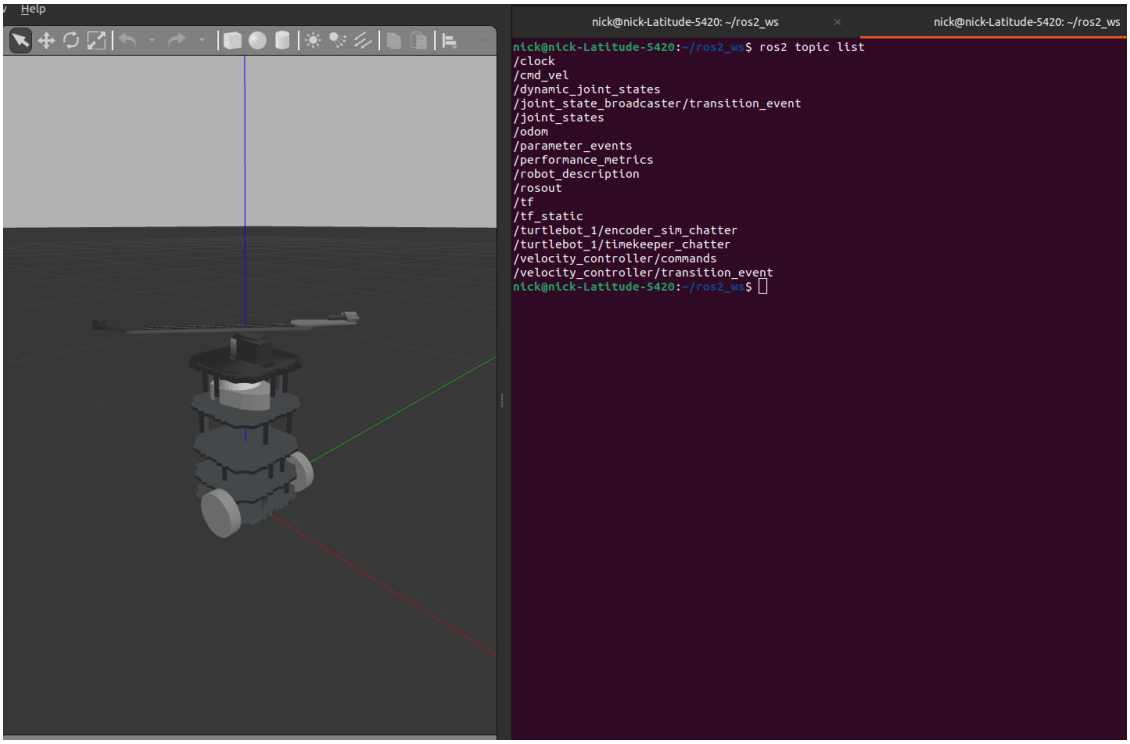


Figure 7: Using the command "ros2 topic list", we can see the active ROS topics running with the Gazebo simulation.

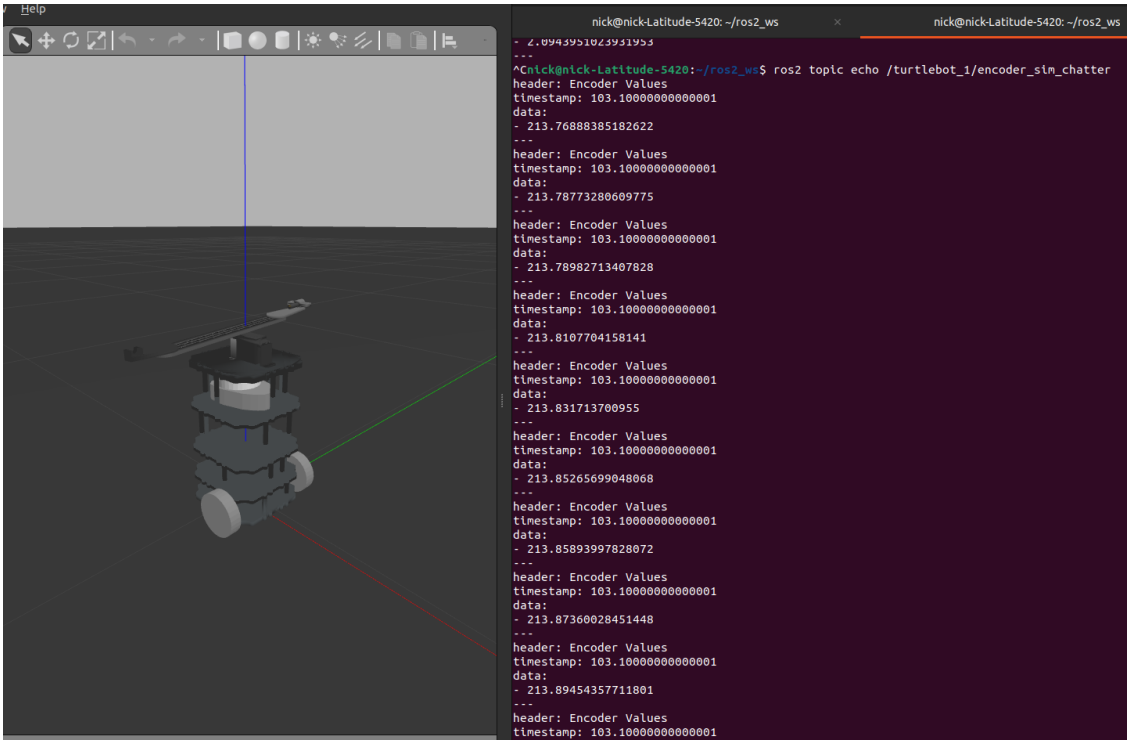


Figure 8: We are viewing what the encoder node is publishing using the command "ros2 topic echo /turtlebot_1/encoder_sim_chatter".

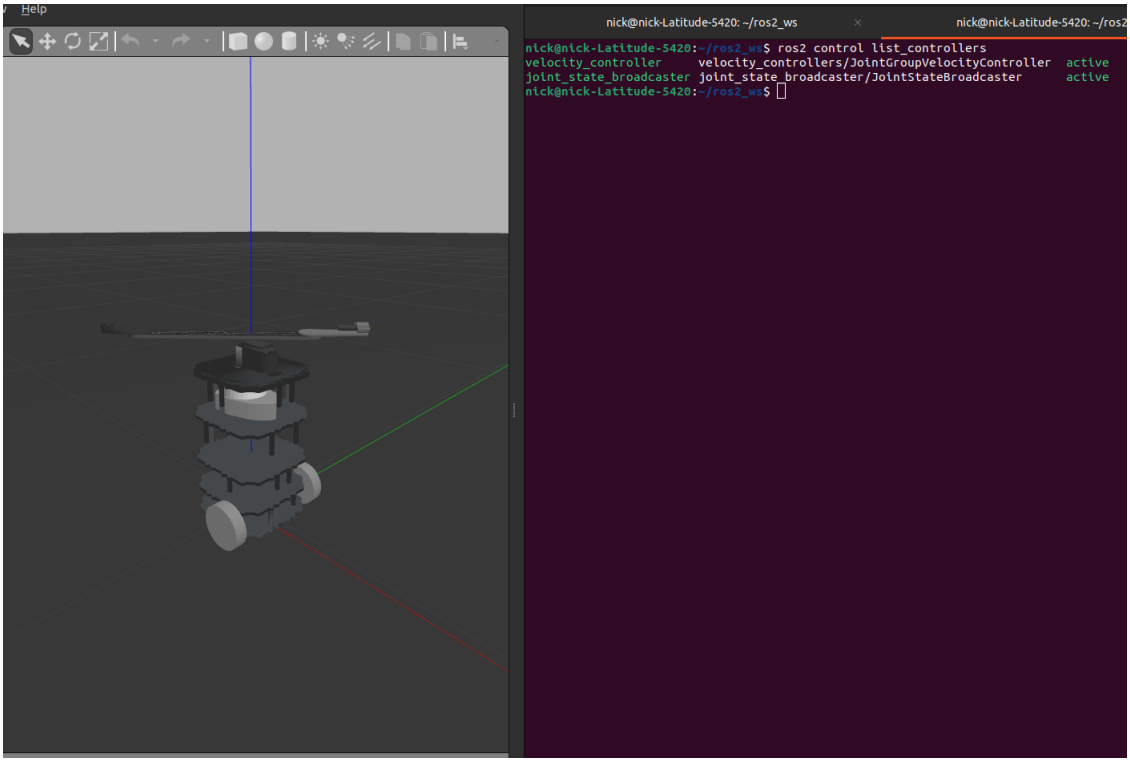


Figure 9: We can view the active Gazebo ROS2 controllers setup in 3.4 using the command "ros2 control list_controllers".

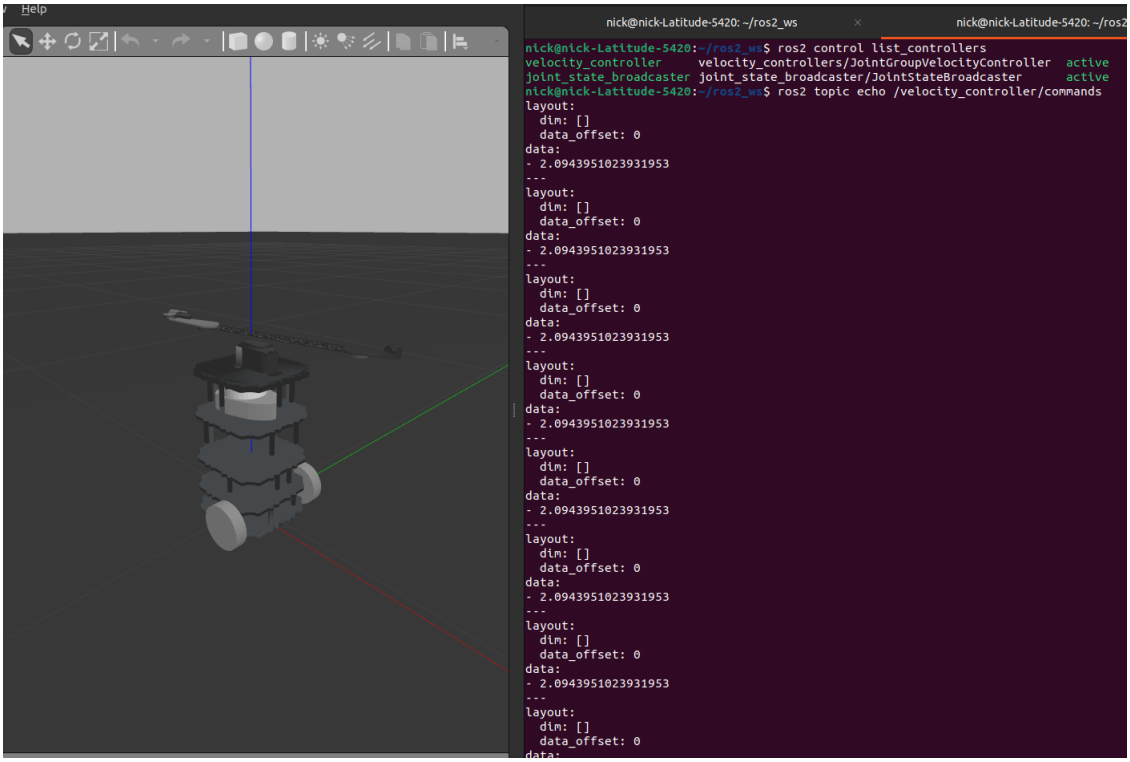


Figure 10: We can view the velocity commands the rotate sensor frame node is publishing with the command "ros2 topic echo /velocity_controller/commands".

The commands "ros2 topic list" and "ros2 topic echo" are extremely useful since they allow a user to track what the ROS nodes are doing in an active simulation. After each of the following sections, I invite the user to use these commands to verify that the following ROS nodes are publishing data correctly.

4.5 Sensor Pose Node

This node is used to obtain the global positions and orientations of sensors placed on rotate frames. This node uses a forward kinematic method to calculate the transformation matrix describing a sensor in the environment. To implement this forward kinematic method, both the position of the vehicle's footprint and the current angular position readings are used.

4.5.1 Forward Kinematic Method

A transformation matrix $T_{a,b}$ describes the transform from a coordinate frame a to a frame b. This matrix has the following form:

$$T_{a,b} = \begin{pmatrix} R & p \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} r_{11} & r_{12} & r_{13} & p_x \\ r_{21} & r_{22} & r_{23} & p_y \\ r_{31} & r_{32} & r_{33} & p_z \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Here, R denotes a rotation matrix which describes the orientation of frame b in frame a, and vector p denotes the position of the origin of frame b in the coordinates of frame a. Transformation matrices like this are used to describe position and orientation of one frame b with respect to another frame a. We will make use of them to describe the position of the sensor frame sensor with respect to a stationary odometry frame odom.

We will make use of the subscript cancellation rule shown below. Note $T_{a,b}$ describes frame b in frame a, and $T_{b,c}$ describes frame c in frame b, thus to describe frame c in frame a, one simply has to do the

multiplication of the transformation matrices in the order shown below.

$$T_{a,b} * T_{b,c} = T_{a,c}$$

This node uses the following forward kinematic equation to calculate sensor position:

$$T_{odom,sensor} = T_{odom,vehicle} * T_{vehicle,r\_joint} * T_{r\_joint,sensor} \tag{1}$$

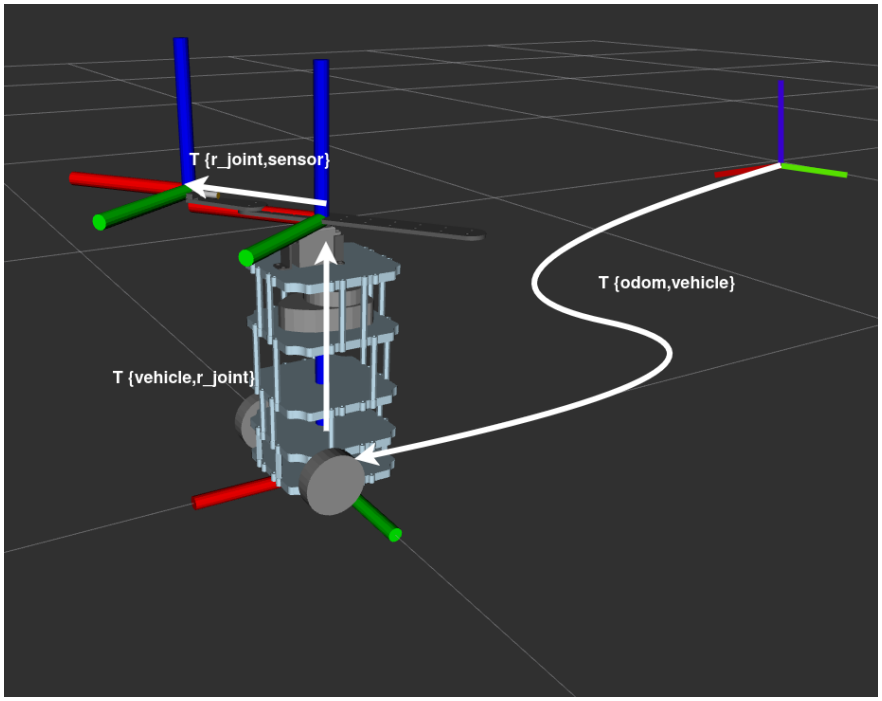


Figure 11: Visualization of transformation matrices for a turtlebot vehicle.

First, the odometry information is used to calculate the pose of the vehicle's footprint frame w.r.t. the fixed odom frame (for a turtlebot, this is set up in the robot's URDF file in the differential drive plugin, other vehicles making use of this package will have to set up something similar). With this, the transformation matrix $T_{odom,vehcile}$ can be created. Note this matrix changes with time, as the robot moves, this matrix gets updated with the odom callback. The odom information contains the vehicle position (px, py, pz) as well as quaternion angles (qw, qx, qy, qz). The position vector can be directly inserted into a transformation matrix, and the quaternion angles can be converted into a rotation matrix. See eqn (7b) from this reference for more information.

$$T_{odom,vehicle} = \begin{pmatrix} 1 - 2 * q_y * *2 - 2 * q_z * *2 & 2 * q_x * q_y - 2 * q_w * q_z & 2 * q_x * q_z + 2 * q_w * q_y & p_x \\ 2 * q_x * q_y + 2 * q_w * q_z & 1 - 2 * q_x * *2 - 2 * q_z * *2 & 2 * q_y * q_z - 2 * q_w * q_x & p_y \\ 2 * q_x * q_z - 2 * q_w * q_y & 2 * q_y * q_z + 2 * q_w * q_x & 1 - 2 * q_x * *2 - 2 * q_y * *2 & p_z \\ 0 & 0 & 0 & 1 \end{pmatrix} \tag{2}$$

Second, the encoder information is used to calculate the transformation matrix $T_{vehicle,rjoint}$. This node constructs $T_{vehicle,rjoint}$ as a lambda function where the position portion of the T matrix is constant, however the rotation matrix portion of T gets updated with every new encoder reading. Note, that $T_{vehicle,rjoint}$ changes with time, as the frame rotates, this matrix gets updated with the encoder callback.

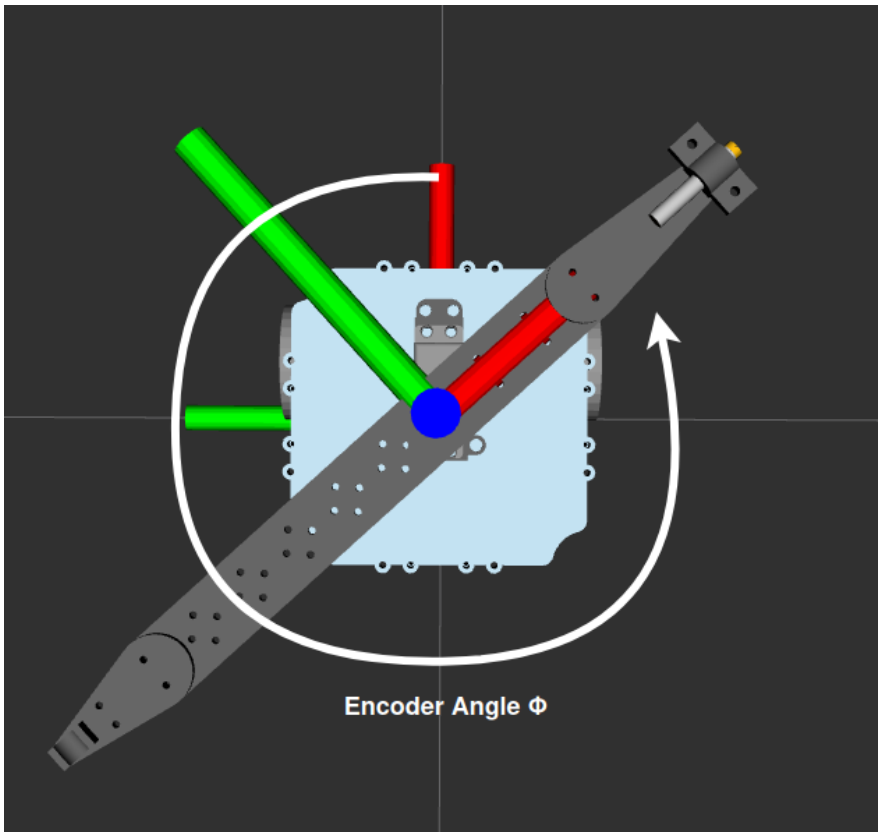


Figure 12: Visualization of the encoder angle with a Turtlebot vehicle.

The rotation axis of this revolute joint and the position of this revolute joint $p = [pa, pb, pc]^T$ w.r.t. the vehicle's footprint are both specified from the json file. We can use equations (4b-e) from this reference to construct quaternion angles knowing the value of our angular position ϕ. These equations are restated below.

$$q_0 = \cos\frac{\phi}{2} \tag{3}$$

$$q_1 = \hat{x}\sin\frac{\phi}{2} \tag{4}$$

$$q_2 = \hat{y}\sin\frac{\phi}{2} \tag{5}$$

$$q_3 = \hat{z}\sin\frac{\phi}{2} \tag{6}$$

We can then use these quaternion angles to construct the $T_{vehicle,r\_joint}$ transformation matrix using the equation below. See equation (7b) from this reference for more information.

$$T_{r\_joint,sensor} = \begin{pmatrix} 1-2q_2^2-2q_3^2 & 2q_1q_2-2q_0q_3 & 2q_1q_3+2q_0q_2 & pa \\ 2q_1q_2+2q_0q_3 & 1-2q_1^2-2q_3^2 & 2q_2q_3-2q_0q_2 & pb \\ 2q_1q_3-2q_0q_2 & 2q_2q_3+2q_0q_1 & 1-2q_1^2-2q_2^2 & pc \\ 0 & 0 & 0 & 1 \end{pmatrix} \tag{7}$$

Third, the $T_{r\_joint,sensor}$ transformation matrix is taken directly from the json configuration file, where $p = [px, py, pz]^T$ is sensor position with respect to the revolute joint. Note this matrix does not change with

time.

$$T_{r\_joint,sensor} = \begin{pmatrix} 1 & 0 & 0 & px \\ 0 & 1 & 0 & py \\ 0 & 0 & 1 & pz \\ 0 & 0 & 0 & 1 \end{pmatrix} \tag{8}$$

4.5.2 Launch Arguments

This sensor pose node expects a few input arguments from the user to launch correctly. These are given in the ordered list below.

1. The user must name an input odometry topic, this is used to collect Odometry messages which contain the position and orientation of the vehicle's footprint in the simulation environment
2. The user must name an input encoder topic, this is used to collect StampedFloat64MultiArray messages which contain the angular positions of the rotating sensor frames.
3. The user must name an input timekeeper topic, this is used to collect Timekeeper messages which contain timekeeping information.
4. The user must name an output topic to publish StampedTransformMultiArray messages to, these messages contain the transformation matrices describing the position and orientation of each sensor in the environment.
5. The user must name a transform configuration file to use, this gives the sensor pose node a configured object to use in an experiment.

These arguments are then used to form the launch command for the sensor pose node seen in 4.5.5.

4.5.3 Transform Config File

The sensor pose node must be configured with a config file, an example of which is seen below. First note the name "rotating_frame_one" at the top of the config file. For every individual rotating sensor frame, we need to configure one individual transform object, the names of these should be distinct to prevent confusion.

The user must supply the absolute file path to a python script to reference, and name an object written in that script to use with this node. The params dictionary is then used to configure that object with items specified by the user. In this example, we select the "Transform_Odom_To_Sensor_Pose" object from the given file path. The params dictionary will configure that object with the joint position, the rotation axis, and the sensor transform keys. The joint position key should refer to an array that describes the position of the rotating sensor frame joint with respect to the vehicle's "footprint" or "base" frame specified in the URDF file. The rotation axis key should be an array containing a unit vector describing the rotation axis of the rotating sensor frame in the local vehicle frame. Finally, the sensor position key should be a transformation matrix that describes the position and orientation of the sensor with respect to the rotation sensor frame joint. This matrix does not change as the frame rotates, this is a static transform.

```
{
    "rotating_frame_one":{
        "filepath": "~/ros2_ws/src/ros_esc/ros_esc/sensor_pose_node/transform_objects/transform_objects.py",
        "object_name": "Transform_Odom_To_Sensor_Pose",
        "params":{
            "joint_position": [0, 0, 0.2932],
            "rotation_axis": [0, 0, 1],
            "sensor_transform": [
                [1, 0, 0, 0.1524],
                [0, 1, 0, 0],
                [0, 0, 1, 0.015],
                [0, 0, 0, 1]
            ]
        }
    }
}
```

More examples of transform config files can be found at the following location.

`~/ros2_ws/src/ros_esc/ros_esc/sensor_pose_node/transform_config_files`

For users who want to test new config files, they can check to make sure they are parsed correctly using the testing script at the following location.

`~/ros2_ws/src/ros_esc/ros_esc/sensor_pose_node/transform_config_testing.py`

4.5.4 Transform Object

This node needs a transform object specified by the user to use in an experiment. During an experiment, the sensor pose node will always give this transform object the same two inputs: an array of odometry data, and an angular position. The node will always expect a transformation matrix output from this transform object.

The above example selects a transform object named "Transform_Odom_To_Sensor_Pose" from within the ROS ESC package. This object can be found within the python script at the following location.

`~/ros2_ws/src/ros_esc/ros_esc/sensor_pose_node/transform_objects/transform_objects.py`

Looking into this python script, we can see that this transform object takes in the parameters from the config file and uses them to create the three transformation matrices described in the forward kinematic method. The sensor pose node will constantly call the "transform_output" method of this object during an active experiment. We can see that this transform output method takes in the odometry data and angular position of the rotating sensor frame, and returns the final transformation matrix at the bottom. For vehicles with multiple rotating sensor frames, the final transformation matrices will be compiled together into the output message.

```
# pylint: disable=too-few-public-methods
# pylint: disable=invalid-name
class Transform_Odom_To_Sensor_Pose(TransformObject):
    """This calculates the transformation matrix of a sensor based on odometry and encoder data."""

    def __init__(self, params):
        """This initializes the transform object.

        The transform odom to sensor pose object should be configured in the config
        file as a dictionary named params in the following format. The joint position
        key should be an array that describes the position of the rotating sensor frame
        joint with respect to the vehicle's 'footprint', or 'base' frame. Note that this
        'footprint' or 'base' frame is specified in the vehicle's URDF file, and can be
        thought of as the vehicle's center of mass. The rotation axis key should be an
        array containing a unit vector describing the rotation axis of the rotating sensor
        frame in the local vehicle frame. The sensor position key should be a transformation
        matrix that describes the position and orientation of the sensor with respect the the
        rotating sensor frame joint. This matrix does not change as the frame rotates,
        this is a static transform.

        "params":{
            "joint_position": np.array (3,) (one row by three columns),
            "rotation_axis": np.array (3,) (one row by three columns),
            "sensor_transform": np.array (4,4) (four rows by four columns)
        }
        """

        # Assert all inputs are arrays of the correct dimensions
        joint_position = params["joint_position"]
        assert np.shape(joint_position) == (3,), "The joint_position must be an array of length three."
        rotation_axis = params["rotation_axis"]
        assert np.shape(rotation_axis) == (3,), "The rotation_axis must be an array of length three."
```

```
        sensor_transform = params["sensor_transform"]
        assert np.shape(sensor_transform) == (4, 4), "The sensor_position must be a 4x4 transformation matrix.
    "

        # Initialize the object with the inputs
        self.joint_position = joint_position
        self.rotation_axis = rotation_axis
        self.sensor_transform = sensor_transform

    # pylint: disable=no-self-use
    def odom_transform_matrix(self, odom_data):
        """This generates the transformation matrix from odom to the vehicle 'footprint'."""

        # The odom data is an array in the form
        # odom_data = [x_pos, y_pos, z_pos, quat_w, quat_x, quat_y, quat_z]

        # Get the vehicle's position
        x_pos = odom_data[0]
        y_pos = odom_data[1]
        z_pos = odom_data[2]

        # Note we have to use the quaternion angles given to us
        # by msg.pose.pose.orientation to calculate our r, p, y angles
        # For reference, see eqns (11a-c):
        # https://danceswithcode.net/engineeringnotes/quaternions/quaternions.html
        quat_w = odom_data[3]
        quat_x = odom_data[4]
        quat_y = odom_data[5]
        quat_z = odom_data[6]

        # Package this position and orientation data into a homogeneous transformation matrix
        # Note this transformation matrix represents the position & orientation of the vehicle
        # in the stationary odometry frame. The rotation matrix was constructed using eqn (7b):
        # https://danceswithcode.net/engineeringnotes/quaternions/quaternions.html
        transform_matrix = np.array([
            [1-2*quat_y**2-2*quat_z**2, 2*quat_x*quat_y-2*quat_w*quat_z,
             2*quat_x*quat_z+2*quat_w*quat_y, x_pos],

            [2*quat_x*quat_y+2*quat_w*quat_z, 1-2*quat_x**2-2*quat_z**2,
             2*quat_y*quat_z-2*quat_w*quat_x, y_pos],

            [2*quat_x*quat_z-2*quat_w*quat_y, 2*quat_y*quat_z+2*quat_w*quat_x,
             1-2*quat_x**2-2*quat_y**2, z_pos],

            [0,0,0,1]
        ])

        return transform_matrix

    def joint_transform_matrix(self, angular_position):
        """This generates the transformation matrix from the vehicle's 'footprint' to the joint."""

        # Convert an axis angle representation to quaternions
        # For reference, see eqns (4b-e):
        # https://danceswithcode.net/engineeringnotes/quaternions/quaternions.html
        quat_w = np.cos(angular_position / 2)
        quat_x = self.rotation_axis[0] * np.sin(angular_position / 2)
        quat_y = self.rotation_axis[1] * np.sin(angular_position / 2)
        quat_z = self.rotation_axis[2] * np.sin(angular_position / 2)

        # Convert these quaternions to a rotation matrix,
        # combine with position information the joint position variable

        # Package this position and orientation data into a homogeneous transformation matrix
        # Note this transformation matrix represents the position & orientation of the rotating
        # sensor frame in the vehicle footprint frame. For reference, see eqn (7b):
        # https://danceswithcode.net/engineeringnotes/quaternions/quaternions.html
        transform_matrix = np.array([
```

```
            [1-2*quat_y**2-2*quat_z**2, 2*quat_x*quat_y-2*quat_w*quat_z,
             2*quat_x*quat_z+2*quat_w*quat_y, self.joint_position[0]],

            [2*quat_x*quat_y+2*quat_w*quat_z, 1-2*quat_x**2-2*quat_z**2,
             2*quat_y*quat_z-2*quat_w*quat_x, self.joint_position[1]],

            [2*quat_x*quat_z-2*quat_w*quat_y, 2*quat_y*quat_z+2*quat_w*quat_x,
             1-2*quat_x**2-2*quat_y**2, self.joint_position[2]],

            [0,0,0,1]
        ])

        return transform_matrix

    def transform_output(self, odom_data, angular_position):
        """This uses the forward kinematic method to get the transformation matrix of the sensor."""

        # Get the transformation matrix from odom to vehicle footprint
        odom_transform = self.odom_transform_matrix(odom_data)

        # Get the transformation matrix from vehicle footprint to joint
        joint_transform = self.joint_transform_matrix(angular_position)

        # Multiply these two transformation matrices together
        intermediate_transform = np.matmul(odom_transform, joint_transform)

        # Multiply this by the sensor transform to get the final transformation
        # matrix describint the sensor's pose in the global frame
        final_transform = np.matmul(intermediate_transform, self.sensor_transform)

        # Return the final transformation matrix describing the sensor
        output = final_transform

        return output
```

More examples of transform objects can be found at the following location.

`~/ros2_ws/src/ros_esc/ros_esc/sensor_pose_node/transform_objects`

4.5.5 Adding Node to Launch File

We now add the following items to the main Gazebo launch file. First, we initialize defaults for the variables necessary to launch this node

```
    <!-- Set the default transform configuration filepath -->
    <arg name="sensor_transform_config_filepath" default="~/ros2_ws/src/ros_esc/ros_esc/sensor_pose_node/
         transform_config_files/turtlebot_rotating_sensor.json"/>
```

We then create a command to launch the rotate sensor frame node with the arguments given in 4.5.2. Note we setup the ROS topic publishing and subscribing with these launch arguments.

```
    <!-- Launch the sensor pose node -->
    <executable cmd="
        ros2 run ros_esc sensor_pose_node
        /odom
        $(var entity_name)/encoder_chatter
        $(var entity_name)/timekeeper_chatter
        $(var entity_name)/sensor_transform_chatter
        $(var sensor_transform_config_filepath)
    "
    />
```

4.6 Cost Function Node

This node is used to calculate the cost values associated with unique sensors based on the their position and orientation in an environment, and the current time. This node allows for cost function expressions, or interpolated maps to be used as an objective function for the seeker vehicle to explore. In addition, the user has the ability to configure a a noise object to add noise to their cost function signal. This gives one the ability to test designs with simulated "sensor noise".

4.6.1 Launch Arguments

This cost function node expects a few input arguments from the user to launch correctly. These are given in the ordered list below.

1. The user must name an input transform topic, this is used to collect StampedTransformMultiArray messages which contain an array of transforms for unique sensors to evaluate with.
2. The user must name an input timekeeper topic, this is used to collect Timekeeper messages which contain timekeeping information to reference.
3. The user must name an output topic to publish StampedTransformMultiArray messages to, these messages contain the cost value data.
4. The user must name a cost function configuration file to use, this gives the cost function node a configured object to use in an experiment.

These arguments are then used to form the launch command for the cost function node seen in 4.6.5.

4.6.2 Cost Function Config File

The cost function node must be configured with a config file, an example of which is seen below. First note the names "CostFunction" and "Noise", within these sections we are configuring a cost function object and a noise object respectively.

The user must supply the absolute file path to a python script to reference, and name an object written in that script to use for both the cost function and noise objects. In both cases, a params dictionary is used to configure the objects with any necessary input arguments. In this example, we select the "Position_Based_Sympy_Expression" object to use as our cost function object. The params dictionary will configure that object with the function, symbols, and substitutions keys. The function key describes the main cost function expression. The symbols key is a list of the base symbols for that main cost function expression. Finally, the substitutions key is used to make any substitutions into the main expression. In this example, we select the "No_Noise" object to use as our noise object. We see that this object does not need any input parameters to configure it.

```
{
    "CostFunction":{
        "filepath": "~/ros2_ws/src/ros_esc/ros_esc/cost_function_node/cost_function_objects/
             cost_function_objects.py",
        "object_name": "Position_Based_Sympy_Expression",
        "params":{
            "function": "a*(x-x_optimal)**2 + a*(y-y_optimal)**2",
            "symbols": ["t", "x", "y", "z"],
            "substitutions":{
                "a": 0.5,
                "x_optimal": 3,
                "y_optimal": 3
            }
        }
    },
    "Noise":{
        "filepath": "~/ros2_ws/src/ros_esc/ros_esc/cost_function_node/cost_function_objects/noise_objects.py",
        "object_name": "No_Noise"
    }
```

```
}
```

More examples of cost function config files can be found at the following location.

`~/ros2_ws/src/ros_esc/ros_esc/cost_function_node/cost_function_config_files`

For users who want to test new config files, they can check to make sure they are parsed correctly using the testing script at the following location.

`~/ros2_ws/src/ros_esc/ros_esc/cost_function_node/cost_function_config_testing.py`

4.6.3 Cost Function Object

This node needs a cost function object specified by the user to use in an experiment. This is described under the "CostFunction" key in the config file. During an experiment, the cost function node will always give this cost function object the same two inputs: the current time, and a transformation matrix describing the position and orientation of an individual sensor. The node will always expect a cost value output from this cost function object.

The config file example selects a cost function object named "Position_Based_Sympy_Expression" from within the ROS ESC package. This object can be found within the python script at the following location.

`~/ros2_ws/src/ros_esc/ros_esc/cost_function_node/cost_function_objects/cost_function_objects.py`

Looking into this python script, we can see that this cost function object takes in the parameters from the config file and uses them to create a sympy expression for a quadratic cost function. The cost function node will constantly call the "cost_output" method of this object during an active experiment. We can see that this cost output method takes in the time and the sensor transformation matrix. We see that it only extracts the position of the sensor from this transformation matrix. It then gives the sensor position and the current time to the sympy expression to evaluate with and calculate a cost value.

```
# pylint: disable=too-few-public-methods
# pylint: disable=invalid-name
class Position_Based_Sympy_Expression(CostFunction):
    """This class allows for a position based sympy expression to be used as a cost function."""

    def __init__(self, params):
        """This initializes the object.

        The sympy expression should be defined in the config file as a dictionary
        named params in the following format. The function key should contain the
        cost function expressed symbolically as a string. The symbols key should
        match the one shown below, all cost functions will be functions of position
        and time. Finally, the substitutions key is optional, this can be used to
        define a dictionary of variables that will get substituted into the main
        expression. These variables may be defined as ints or floats, or they may
        contain more expressions expressed as strings.

        "params":{
            "function": {cost_function_expressed_symbolically_as_a_string},
            "symbols": ["t", "x", "y", "z"],
            "substitutions":{
                "substitution_1": float
                "substitution_2": {expression_string}
                etc...
            }
        }
        """

        # Create a lambda function with sympy
        # pylint: disable=unused-variable
        self.cost_function, self.cost_expression = parse_sympy_expression(params)

```

```
    def cost_output(self, time, sensor_transform):
        """This operates on input arguments and produces a cost value."""

        # Extract the sensor's position from the transformation matrix
        x = sensor_transform[0][3]
        y = sensor_transform[1][3]
        z = sensor_transform[2][3]

        # Calculate the cost value with the expression
        output = self.cost_function(time, x, y, z)

        return output
```

More examples of cost function objects can be found at the following location.

`~/ros2_ws/src/ros_esc/ros_esc/cost_function_node/cost_function_objects`

4.6.4 Noise Object

This node needs a noise object specified by the user to use in an experiment. This is described under the "Noise" key in the config file. During an experiment, the cost function node will always give this noise object the same two inputs: the current time, and a cost value calculated with the previous cost function object. The node will always expect an altered cost value output from this noise object.

The config file example selects a noise object named "No_Noise" from within the ROS ESC package. This object can be found within the python script at the following location.

`~/ros2_ws/src/ros_esc/ros_esc/cost_function_node/cost_function_objects/noise_objects.py`

Looking into this python script, we see that the cost function node will constantly call the "add_noise" method of this object during an active experiment. We can see that this add noise method takes in the time and the cost value and returns the altered cost value output. However this "No_Noise" object does not apply any noise to the cost value at all, it simply returns the value it was given. This "No_Noise" object should be considered the default choice for a noise object for most experiments. Other noise objects that do alter the cost value in some way would only be used for more advanced experimentation.

```
# pylint: disable=too-few-public-methods
# pylint: disable=invalid-name
class No_Noise(NoiseObject):
    """This class implements no noise, it is simply a placeholder in a config file."""

    def __init__(self):
        """This initializes the object."""

        # pylint: disable=unnecessary-pass
        pass

    def add_noise(self, time, cost):
        """This takes in the cost value and adds in noise."""

        # Return the unaltered cost values
        output = cost
        # Convert from array to list
        output = output.tolist()

        return output
```

More examples of noise objects can be found at the following location.

`~/ros2_ws/src/ros_esc/ros_esc/cost_function_node/cost_function_objects/noise_objects.py`

4.6.5 Adding Node to Launch File

We now add the following items to the main Gazebo launch file. First, we initialize defaults for the variables necessary to launch this node

```
<!-- Set the default cost function configuration filepath-->
<arg name="cost_function_config_filepath" default="~/ros2_ws/src/ros_esc/ros_esc/cost_function_node/
    cost_function_config_files/static_2D_quadratic.json"/>
```

We then create a command to launch the rotate sensor frame node with the arguments given in 4.6.1. Note we setup the ROS topic publishing and subscribing with these launch arguments.

```
<!-- Launch the cost function node -->
<executable cmd="
    ros2 run ros_esc cost_function_node
    $(var entity_name)/sensor_transform_chatter
    $(var entity_name)/timekeeper_chatter
    $(var entity_name)/cost_value_chatter
    $(var cost_function_config_filepath)
"
/>
```

4.7 Filter Node

This node is used to conduct any filtering needed for derivative estimation using a custom filter made using the extremum seeking package in section 2.3.5. This node allows for custom filter designs made of filter and parameter ODE objects from the extremum seeking package to be used to operate on cost value data. In addition, it is often the case for vehicles with rotating sensors that angular position data from the encoder node is needed. This angular position data plays a critical role in forming estimation signals which are convolved with cost value data to estimate local derivatives. The user has the option to concatenate angular position data of the rotating sensor with the cost value data, and the combined input will be given to the custom filter.

4.7.1 Custom Filter Design

The idea behind specifying a custom filter's architecture in the following form is to allow for a user to save and share configurations which enables rapid testing and deployment. This filter node does not hard code anything, rather it expects a user to fully describe what their desired filter looks like. The filter node is responsible for parsing the configuration file, creating the custom filter as described, and using it with ROS2 communication to operate on input values.

Each filter in the configuration file (with the exception of Cascade and Parallel filters) must be written as a dictionary. The elements contained within the dictionary, called keys, can contain things like gains, filter dimensions, functions, symbols etc. Each key must have a string as a name to indicate what the following value means. Typically these names come from the required inputs for that particular filter in the extremum-seeking package, however more complicated filters like the ConvolutionFilter, FunctionFilter, and DirectionalFilter, require additional keys to fully describe and initialize the filter. Note for single input, single output (SISO) filters, the dictionary keys would refer to values of type int or float. However, for multi input multi output (MIMO) systems, the dictionary keys may refer to lists of ints or floats, which may correspond to multiple gains.

Cascade and Parallel filters are aggregate filters, meaning they are built up one or more subfilters. When describing these filters in a configuration file, note they are written as a list of dictionaries separated by commas i.e. [dict_1, dict_2, ...]. Think of this as a list of several sub filters, which will be combined together in an aggregate filter. Cascade filters combine all the sub filters in series in the order they appear in the file. Parallel filters combine all sub filters in parallel. There is no limit to the amount of sub filters contained in these aggregate filters.

Some filters, like Convolution and Function filters may contain user defined functions to operate with. In this case, several keys are required for the filter node to correctly create the filter object. A "function" key is required for the user to define their custom function for the filter to use, this function must be written as a string. A "symbols" key may be used to define the base symbol the function uses. For a function defined as f(t), this base symbol would be "t", however for a function defined f(y), the symbol would be defined as "y". Note if a symbols key is used in a function filter, the equation will be parsed with sympy parse expression, then converted to a lambda function using sympy lambdify. However if a function is defined with no symbols key, the expression will be parsed with the eval() function. There are subtle differences in parsing functions between these two methods that are not covered here. Finally, an optional "substitutions" key is used to define additional symbols and expressions that are substituted into the main function, all of these written out in another dictionary. This key allows for the user to build up complex functions from smaller expressions. One must note however that any additional variables introduced in the substitutions dictionary must either be constants, or somehow dependent on the base variable defined by "symbols".

To use functions in the MIMO case, these "function", "symbols", and "substitutions" keys must now refer to lists where the indices correlate with one another. In this case, "function" refers to a list of strings representing expressions, "symbols" refers to a list of strings representing the base symbol for each expression, and "substitutions" refers to a list of dictionaries, where each dictionary contains the substitutions for one expression. See example 4.3 for more details.

For users who would like to test these examples on their own, I recommend copying these example configs into .json files and testing them with the config testing script in the following location.

```
~/ros2_ws/src/ros_esc/ros_esc/filter_node/filter_config_testing.py
```

Note the user may have to adjust some of the inputs to these custom filters to ensure the script runs correctly and produces a result.

Example 4.1

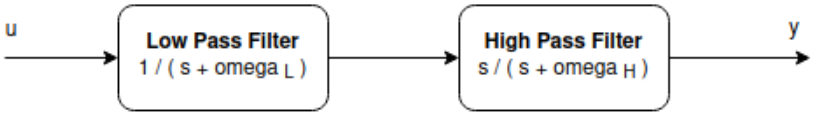


Figure 13: Example filter 1.

```
{
    "CascadeFilter":[
        {
            "LowPassFilter":{"omega_l": 1.0, "init_states": [1.0]}
        },
        {
            "HighPassFilter":{"omega_h": 0.5}
        }
    ]
}
```

This example uses a CascadeFilter to put a list of two subfilters, LowPassFilter and HighPassFilter, into a series connection. Both of these subfilters are SISO, with gains specified in their respective dictionaries. The CascadeFilter puts the subfilters in the serial connection in the order they appear, i.e. the input to the custom filter passes through the LPF first, then the HPF. Note that the LowPassFilter is given an initial state array to use, while the HighPassFilter is not. Any filter with internal dimensions that is not given

any initial states will be initialized with an appropriate amount of zeros, however this can be overriden with custom initial states using the "init_states" key as seen here. Once this filter is parsed, the vector of the custom filter initial states will be [1, 0].

Example 4.2

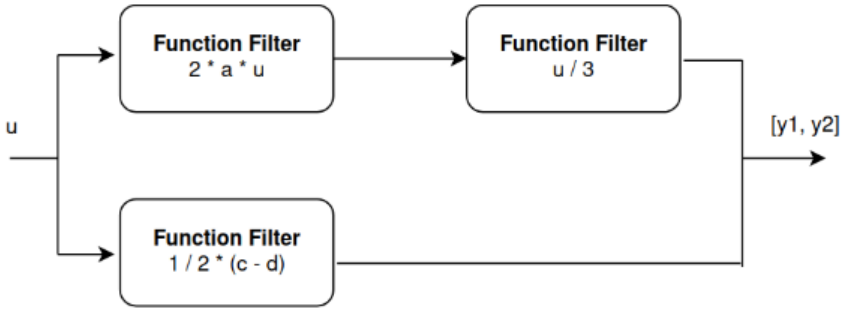


Figure 14: Example filter 2.

```
{
    "ParallelFilter":[
        {
            "CascadeFilter":[
                {
                    "FunctionFilter":
                    {
                        "function": "2*a*u",
                        "symbols": "u",
                        "substitutions": {"a": 4},
                        "idim": 1,
                        "odim": 1
                    }
                },
                {
                    "FunctionFilter":
                    {
                        "function": "u/3",
                        "symbols": "u",
                        "idim": 1,
                        "odim": 1
                    }
                }
            ]
        },
        {
            "FunctionFilter":
            {
                "function": "1/2*(c-d)",
                "symbols": "u",
                "substitutions": {"c": 4, "d": "1/2*u"},
                "idim": 1,
                "odim": 1
            }
        }
    ]
}
```

This example puts uses a ParallelFilter to put a list of two subfilters, CascadeFilter and FunctionFilter, into a parallel connection, meaning they both receive the same input. Note the use of the "function", "symbols", and "substitutions" keys in the function filters. Each expression in "function" is written as a string. Each symbol in "symbols" is a string representing the base variable (u in this case). Finally note that some filters make use of the "substitutions" key, while others do not. Note in the bottom FunctionFilter, the base symbol "u" does not appear directly in the "function" expression, rather it comes up through the substitutions. Note that this base symbol could be any string, meaning these filters could have been written with base symbols "y", "phi", "theta" etc. Substitutions may be used to sub in constants, or used to sub in additional expressions written as strings.

Example 4.3

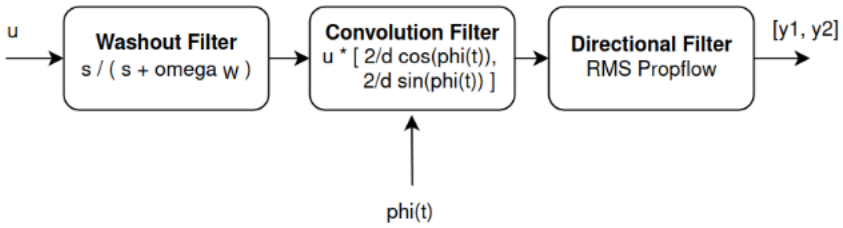


Figure 15: Example filter 3.

```
{
    "CascadeFilter":[
        {
            "WashoutFilter":{
                "omega": 1.0
            }
        },
        {
            "ConvolutionFilter":{
                "function": ["2/d*cos(phi)","2/d*sin(phi)"],
                "symbols": ["t","t"],
                "substitutions": [
                    {"d": 0.1524, "phi": "phi0 + omega*t", "phi0": 0, "omega": "2*pi/60*rpm", "rpm": 20},
                    {"d": 0.1524, "phi": "phi0 + omega*t", "phi0": 0, "omega": "2*pi/60*rpm", "rpm": 20}
                ],
                "odim": 2
            }
        },
        {
            "DirectionalFilter":{
                "param_ode": "RMSpropFlow",
                "k": [0.1,0.1],
                "omega_l": 1.0
            }
        }
    ]
}
```

In this example, the CascadeFilter is used to put a WashoutFilter, a ConvolutionFilter, and a DirectionalFilter into a series connection. In the ConvolutionFilter, we again see the "function", "symbols", and "substitutions" keys appear, however now they are used in the MIMO case. Each of these keys refer to lists where the indicies correlate with one another. Note that a ConvolutionFilter must have the base symbol "t", this is required unlike the FunctionFilter. Additionally, note in these substitutions dictionaries, the substitutions can be written out in any order. As long as everything is accounted for, the filter node will parse it

down to a final expression. In the DirectionalFilter, a "param_ode" is specified. This refers to a parameter update ODE found in the extremum-seeking package. The additional keys: "k" and "omega_l" are used to initialize the parameter update ODE object. Note in this case, "k" is a list so this DirectionalFilter is MIMO.

Example 4.4

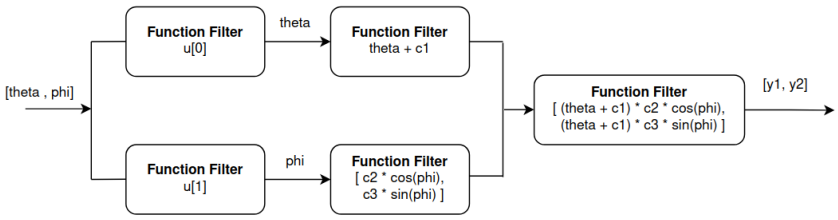


Figure 16: Example filter 4.

```
{
    "CascadeFilter":[
        {
            "ParallelFilter":[
                {
                    "CascadeFilter":[
                        {
                            "FunctionFilter":{
                                "function": "u[0]",
                                "idim": 1,
                                "odim": 1
                            }
                        },
                        {
                            "FunctionFilter":{
                                "function": "theta + c1",
                                "symbols": ["theta"],
                                "substitutions": [
                                    {
                                        "c1": "pi/2"
                                    }
                                ],
                                "idim": 1,
                                "odim": 1
                            }
                        }
                    ]
                },
                {
                    "CascadeFilter":[
                        {
                            "FunctionFilter":{
                                "function": "u[1]",
                                "idim": 1,
                                "odim": 1
                            }
                        },
                        {
                            "FunctionFilter":{
                                "function": [
                                    "c2 * cos(phi)",
```

```
                                "c3 * sin(phi)"
                            ],
                            "symbols": ["phi","phi"],
                            "substitutions": [
                                {
                                    "c2": "pi/4"
                                },
                                {
                                    "c3": "pi"
                                }
                            ],
                            "idim": 1,
                            "odim": 2
                        }
                    }
                ]
            }
        ]
    },
    {
        "FunctionFilter":{
            "function": [
                "u[0]*u[1]",
                "u[0]*u[2]"
            ],
            "idim": 3,
            "odim": 2
        }
    }
]
}
```

The user may run into a case where they need to demux and mux signals in their custom filter. This is accomplished by clever use of ParallelFilters and FunctionFilters. In this example, the input to this filter is assumed to be an array with two elements given by

$$CustomFilterInput = [\theta, \phi]$$

The ParallelFilter along with the first FunctionFilter on each parallel track are used to demux the input signal. The function $u[0]$ *will simply return the first entry of the custom filter input,* θ*, similarly* $u[1]$ *will return* ϕ*. Now we have one track of the ParallelFilter where each subsequent filter in series will operate on the value* θ *and another track where the same happens for* ϕ*. We use more FunctionFilters to perform some mathematical operations on these values.*

When we want to mux these signals back together, we use another FunctionFilter immediately following the end of the ParallelFilter configuration. In this example, this "muxing" FunctionFilter takes in three inputs, one coming from the θ *track, and the other two coming from the* ϕ *track of the of the ParallelFilter. The following equation multiplies the result from the* θ *track by each result of the* ϕ *track. This example demonstrates how one can construct mathematical equations using this demuxing and muxing technique with filter blocks.*

$$CustomFilterOutput = [(\theta + pi/2) * (pi/4 * cos(\phi)), (\theta + pi/2) * (pi * sin(\phi))]$$

This example is useful because this demuxing and muxing technique is used when working with estimationsignals for ESC control. Normally, one would convolve with time varying estimation signals like the ones shown in the ConvolutionFilter in Example 3. Note that these estimation signals are functions of the perturbation signal $\phi(t)$*. This perturbation signal physically represents the angular position of the rotating sensor frame on the vehicle. Estimating this* $\phi(t)$ *signal by explicitly writing it out as a function of time works fine for simple math simulations. However when working with Gazebo or real life vehicles, estimating* $\phi(t)$ *with the ConvolutionFilter introduces timing issues since the actual angular position may differ from the estimated* $\phi(t)$*. This the error will build up as time goes on and make the ESC controller useless.*

It is much better to "convolve" with the actual angular position rather than an estimate by using this demuxing and muxing technique demonstrated in Example 4. The "–encoder_data" option for this filter node allows the user to concatenate the angular position readings from the encoder node with the normal inputs to the filter coming from the input topic. For one sensor and one rotating frame, the custom filter input may look something like the following

$$CustomFilterInput = [J, \phi]$$

The demuxing technique should be used to isolate the cost value J on one track of the parallel filter, while the other track operates on the isolated ϕ signal. This way, any filtering or operations that need to be applied to these specific singals can be kept separate. For example, the mathematical operations that are used to generate the estimation signals all happen on the ϕ track, and dont affect anything happening to the cost value on the J track. Finally, to "convolve" the estimation signals with the result on the J track, one can mux these two signals together with a FunctionFilter as shown in this example above.

4.7.2 Launch Arguments

This filter node expects a few input arguments from the user to launch correctly. These are given in the ordered list below.

1. The user must name an input cost value topic, this is used to collect StampedFloat64MultiArray messages which contain an array of cost value data.
2. The user must name an input encoder topic, this is used to collect StampedFloat64MultiArray messages which contain angular position data for the rotating sensor frames, this may be concatenated with the cost value data as an input to the custom filter if the user desired.
3. The user must name an input timekeeper topic, this is used to collect Timekeeper messages which contain timekeeping information to reference.
4. The user must name an output topic to publish StampedFloat64MultiArray messages to, these contain the output data of the custom filter.
5. Using the "–filter_file" argument, the user must name a filter configuration file to use, this gives the filter node a custom filter object to use in an experiment.
6. Using the "–encoder_data" argument, the user tells the filter node they would like encoder data to be concatenated with cost value data, and the combined array to be inputted into the custom filter. If this argument is not used, only the cost values are given to the custom filter.

These arguments are then used to form the launch command for the filter node seen in 4.7.4.

4.7.3 Filter Config File

The filter node must be configured with a config file, an example of which is seen below. A filter config file is unique from the other types of config files, think of it like a recipe for a custom filter made of ingredients from the extremum seeking package. Within a filter config file, the user must name filter and parameter ODE objects and configure them with the appropriate inputs such as gains, dimensions, functions, omegas, etc.

Once the config file is converted to a custom filter object, it will be used continuously by the filter node. During an experiment, the filter node will always give the custom filter object an array of input values. By default this input array is the cost values that come from the cost function node. However the user does have the option to concatenate the input array with angular position data from the encoder node as well. In this case the cost values come first in the input array, and the angular positions come after. The filter node will always expect an array of output values from this custom filter.

In this example, we have told the filter node that we do want encoder data to be concatenated with the cost value data. The input to this custom filter is an array $u = [J, \phi]$ where J is the cost value, and ϕ is the

concatenated angular position value. This filter has two tracks, the first one only passes the cost value (with function $u[0]$). The second track only passes the angular position value ϕ (with function $u[1]$). In the cost value track, put our cost value J through two separate washout filters to get the result to attenuate mean components of the signal, these two results will be used to estimate the gradient and estimate the squared gradient. In the angular position track, we use ϕ to compute two derivative estimation signals, and two squared gradient estimation signals, these are all defined as functions with symbols and substitutions. Near the bottom, we then convolve the results of the parallel filter together with a function filter that combines the elements together to form gradient and gradient squared estimates. This result is then given to the directional filter using the RMSpropFlow parameter ODE. The result of this is the custom filter output which will then be published.

```
{
   "CascadeFilter":[
      {
         "ParallelFilter":[
            {
               "CascadeFilter":[
                  {
                     "FunctionFilter":{
                        "function": "u[0]",
                        "idim": 1,
                        "odim": 1
                     }
                  },
                  {
                     "ParallelFilter":[
                        {
                           "WashoutFilter":{
                              "omega": 1.0
                           }
                        },
                        {
                           "CascadeFilter":[
                              {
                                 "WashoutFilter":{
                                    "omega": 1.0
                                 }
                              },
                              {
                                 "FunctionFilter":{
                                    "function": "u[0]**2",
                                    "idim": 1,
                                    "odim": 1
                                 }
                              }
                           ]
                        }
                     ]
                  }
               ]
            },
            {
               "CascadeFilter":[
                  {
                     "FunctionFilter":{
                        "function": "u[1]",
                        "idim": 1,
                        "odim": 1
                     }
                  },
                  {
                     "FunctionFilter":{
                        "function": [
                           "2/d*cos(phi)",
                           "2/d*sin(phi)",
                           "(1/d**2)*(3 - 4*(sin(phi))**2)",
```

```
                                "(1/d**2)*(2*sin(2*phi))",
                                "(1/d**2)*(4*(sin(phi))**2 - 1)"
                            ],
                            "symbols": ["phi", "phi", "phi", "phi", "phi"],
                            "substitutions": [
                                {
                                    "d": 0.18
                                },
                                {
                                    "d": 0.18
                                },
                                {
                                    "d": 0.18
                                },
                                {
                                    "d": 0.18
                                },
                                {
                                    "d": 0.18
                                },
                                {
                                    "d": 0.18
                                }
                            ],
                            "idim": 1,
                            "odim": 5
                        }
                    }
                ]
            }
        ]
    },
    {
        "FunctionFilter":{
            "function": [
                "u[0]*u[2]",
                "u[0]*u[3]",
                "u[1]*u[4]",
                "u[1]*u[5]",
                "u[1]*u[6]"
            ],
            "idim": 7,
            "odim": 5
        }
    }
]
}
```

More examples of filter config files can be found at the following location.

```
~/ros2_ws/src/ros_esc/ros_esc/filter_node/filter_config_files
```

For users who want to test new config files, they can check to make sure they are parsed correctly using the testing script at the following location.

```
~/ros2_ws/src/ros_esc/ros_esc/filter_node/filter_config_testing.py
```

4.7.4 Adding Node to Launch File

We now add the following items to the main Gazebo launch file. First, we initialize defaults for the variables necessary to launch this node

```
    <!-- Specify if the filter needs angular position information
    from the encoder node appended to the cost values -->
    <arg name="input_encoder_data_to_filter" default="True"/>

    <!-- Set the default filter configuration filepath -->
```

```
    <arg name="filter_config_filepath" default="~/ros2_ws/src/ros_esc/ros_esc/filter_node/filter_config_files/
        turtlebot_vehicle/adaptive_methods/rmsprop_filter_full_rotation.json"/>
```

We then create a command to launch the rotate sensor frame node with the arguments given in 4.7.2. Note we setup the ROS topic publishing and subscribing with these launch arguments.

```
    <!-- Launch the filter node -->
    <executable cmd="
        ros2 run ros_esc filter_node
        $(var entity_name)/cost_value_chatter
        $(var entity_name)/encoder_chatter
        $(var entity_name)/timekeeper_chatter
        $(var entity_name)/filter_value_chatter
        --filter_file
        $(var filter_config_filepath)
        --append_encoder_data
        $(var input_encoder_data_to_filter)
    "
    />
```

4.8 Controller Node

This node is used to create a custom controller to operate on input values and command the velocity of the vehicle. The controller can be initialized with gains and parameters, and can be designed to apply appropriate control constraints to the vehicle. The controller will output commanded linear and angular velocities in the following form.

$$velocity\ command = [v_x, v_y, v_z, \omega_x, \omega_y, \omega_z] \tag{9}$$

4.8.1 Launch Arguments

The controller node expects a few input arguments from the user to launch correctly. These are given in the ordered list below.

1. The user must name an input value topic, this is used to collect StampedFloat64MultiArray messages which contain an array of input values to operate with. Typically these input values come directly from the output of a custom filter.

2. The user must name an input state topic, this is used to collect Odometry messages that will be converted to vehicle states (x, y, z and roll, pitch, yaw) and passed as an input to the controller.

3. The user must name an input timekeeper topic, this is used to collect Timekeeper messages which contain timekeeping information to reference.

4. The user must name an output control topic to publish StampedFloat64MultiArray messages to, these messages contain the controller output data with a timestamp.

5. The user must name an output twist topic to publish Twist messages to, these messages are the ones actually used to drive the vehicle with the appropriate plugin. Note these messages do not contain timestamps.

6. The user must name a controller configuration file to use, this gives the controller node a configured object to use in an experiment.

These arguments are then used to form the launch command for the controller node seen in 4.8.4.

4.8.2 Controller Config File

The controller node must be configured with a config file, an example of which is seen below. The user must supply the absolute file path to a python script to reference, and name an object written in that script to use for the controller object. A dictionary for controller parameters and a dictionary for controller gains are both used to configure that controller object. In this example, we select the "Directional_Controller" to use as our controller object. The params dictionary will configure the controller with some constraints we impose for the Turtlebot3 vehicle. The gains dictionary is used to set values for forward and angular velocity gains.

```
{
    "filepath": "~/ros2_ws/src/ros_esc/ros_esc/controller_node/controller_objects/turtlebot_vehicle.py",
    "object_name": "Rotating_Frame_Directional_Controller",
    "gains":{
        "k_vx": 1.0,
        "k_wz": 5.0
    },
    "dynamic_states": {
        "m" : 2,
        "ode" : {
            "filepath" : "~/ros2_ws/src/ros_esc/ros_esc/controller_node/controller_objects/
                turtlebot_ode_objects.py",
            "object_name" : "RMSPropODE",
            "omega_l" : 1.0,
            "k" : [0.1, 0.1],
            "epsilon" : 0.1
        },
        "initial_values": [0, 0]
    },
    "params":{
        "wheel_radius": 0.033,
        "wheel_distance": 0.158,
        "wheel_max_rpm": 70,
        "set_max_vx": 0.1,
        "set_max_wz": 0.5
    }
}
```

More examples of controller config files can be found at the following location.

`~/ros2_ws/src/ros_esc/ros_esc/controller_node/controller_config_files`

For users who want to test new config files, they can check to make sure they are parsed correctly using the testing script at the following location.

`~/ros2_ws/src/ros_esc/ros_esc/controller_node/controller_config_testing.py`

4.8.3 Controller Object

This node needs a controller object specified by the user to use in an experiment. During an experiment, the controller node will always give the controller object the same three inputs: the current time, a vector of the vehicle's states, and input values to operate on that typically come from a filter. The node will always expect a vector of control values as an output from this controller object.

The config file example selects a controller object named "Directional_Controller" from within the ROS ESC package. This object can be found within the python script at the following location.

`~/ros2_ws/src/ros_esc/ros_esc/controller_node/controller_objects/turtlebot_vehicle.py`

Looking into this python script, we can see that this controller object takes in the parameters from the config file to initialize its gains, dynamic states, and control constraints. The controller node will constantly call the "controller_output" method of this object during an active experiment. We can see that this controller output method does not make use of the time or the vehicle states, it only formulates the input values as forward and angular velocity terms, and ensures they abide by the control constraints.

```python
class Rotating_Frame_Directional_Controller(ControllerObject):
    """This can be used with derivative estimations obtained in the vehicle relative frame.

    This class can be used for methods that collect a time history of a derivative estimation.
    Adaptive methods like RMSProp and AdaGrad ESC obtain and track the time history of derivative
    estimates in the vehicle relative frame. These must be converted via a 2D rotation matrix so
    that the derivative estimates are transformed into the absolute frame. The selected ODE can be
    used with the derivative estimates to obtain a direction of travel in the absolute frame
    where one sees improvement in their cost value. This direction must then be converted
    back into the vehicle relative frame. With this result, one can then apply their feedback
    control law to navigate along the desired direction in the relative frame.
    """

    def __init__(self, gains, dynamic_states, params):
        """This initializes the rotating frame directional controller object.

        The gains input should be a dictionary with the following keys
        "gains":{
            "k_vx": float,
            "k_wz": float
        },

        The dynamic states input should be a dictionary with the following keys
        "dynamic_states": {
            "m" : int > 0,

            "ode" : {
                "filepath" : string,
                "object_name" : string,

                    ode_object must have a method: "differential_equation"
                    with call signature: differential_equation(t, x, z, u)

                    t is a float for time
                    x is a vector of vehicle states
                    z is a vector of controller dynamic states (for choice of ODE)
                    u is a vector of input values coming from derivative estimation filter

                Include other keys used to intialize the ODE object such as:
                "omega_l" : float, low pass filter parameter
                "k" : ndarray, gains for the ode equation
                "epsilon" : float, small value in ODE equation
            },

            "initial_values": list[float]
                is a vector of initial controller dynamic states
        }

        The params input should be a dictionary with the following keys
        "params":{
            "wheel_radius": float,
            "wheel_distance": float,
            "wheel_max_rpm": float,
            "set_max_vx": null,
            "set_max_wz": null
        }

        The user can override the maximum forward and angular velocity
        constraints by replacing null with a float value.
        """

        # Initialize a vector of dynamic states
        self.num_dynamic_states = dynamic_states["m"]
        self.dynamic_states = np.zeros((self.num_dynamic_states,))

        # If given initial values for the dynamic states
        if dynamic_states["initial_values"]:
```

```
            # Initialize with user defined initial states
            self.dynamic_states = dynamic_states["initial_values"]

        # Obtain the dictionary describing the ode
        ode_dict = dynamic_states["ode"]
        # Parse this ode dictionary and return the initialized ODE object
        self.ode_object = parse_object_config(ode_dict)

        # Define a variable to track the previous timestamp
        self.prev_tstamp = 0

        # Define forward velocity gain
        self.k_vx = gains["k_vx"]
        # Define angular velocity gain
        self.k_wz = gains["k_wz"]
        # Define necessary parameters
        radius = params["wheel_radius"] # wheel radius in meters
        dist = params["wheel_distance"] # horizontal distance between wheels in meters
        max_rpm = params["wheel_max_rpm"] # maximum no load speed for the driving servo motors

        # Calculate the maximum angular velocity of the driving servos
        omega_max = max_rpm/60*2*np.pi # max angular velocity in rad/s
        # Calculate the maximum velocity constraints
        self.max_vx = omega_max*radius # maximum forward velocity
        self.max_wz = 2*omega_max*radius/dist # maximum turning velocity while turning in place

        # Override the maximum forward velocity if the user desires
        if params["set_max_vx"] is not None:
            self.max_vx = params["set_max_vx"]

        # Override the maximum angular velocity if the user desires
        if params["set_max_wz"] is not None:
            self.max_wz = params["set_max_wz"]

    def update_dynamic_states(self, time, state, input_values, dt):
        """Updates controller dynamic states for choice of ODE method."""

        # Use the ODE object to operate on input information
        diff_eqn_output = self.ode_object.differential_equation(
            time, state, self.dynamic_states, input_values
        )

        # Separate the dictionary into the different parts
        theta_dot = diff_eqn_output["theta_dot"]
        z_dot = diff_eqn_output["z_dot"]

        # Update dynamic states with a forward Euler step
        self.dynamic_states += z_dot * dt

        # Update the previous timestamp
        self.prev_tstamp = time

        # Return the desired update direction in the vehicle relative frame
        return theta_dot

    def controller_output(self, time, state, input_values):
        """This operates on the input arguments and produces the controller output.

        time (float): current time
            (either simulation or real time)
        state (np.ndarray): (6,) vector of system states
            (x, y, z, roll, pitch, yaw)
        input_values (np.ndarray): a vector of input values to operate on
            (typically these come from the output of a filter)
        """

        # Calculate the change in time from previous timestamp
        dt =  time - self.prev_tstamp
```

```
        # Update controller dynamic states,
        # obtain the update direction in the vehicle relative frame
        update_direction = self.update_dynamic_states(time, state, input_values, dt)

        # Calculate the feed back law:
        # Linear velocity vx
        v_x = self.k_vx * update_direction[0]
        # Angular velocity wz
        w_z = self.k_wz * update_direction[1]

        # Apply vehicle constraints:
        # Make sure these commands don't go over our vehicle constraints
        if np.abs(v_x) > self.max_vx:
            v_x = float(self.max_vx * np.sign(v_x))
        if np.abs(w_z) > self.max_wz:
            w_z = float(self.max_wz * np.sign(w_z))

        # Return a list of commanded velocities
        # [vx, vy, vz, wx, wy, wz]
        output = np.array([v_x, 0, 0, 0, 0, w_z])

        return output
```

More examples of controller objects can be found at the following location.

`~/ros2_ws/src/ros_esc/ros_esc/controller_node/controller_objects`

4.8.4 Adding Node to Launch File

We now add the following items to the main Gazebo launch file. First, we initialize defaults for the variables necessary to launch this node

```
    <!-- Set the default controller configuration filepath -->
    <arg name="controller_config_filepath" default="~/ros2_ws/src/ros_esc/ros_esc/controller_node/
         controller_config_files/turtlebot_vehicle/adaptive_methods/rmsprop_controller_full_rotation.json"/>
```

We then create a command to launch the rotate sensor frame node with the arguments given in 4.8.1. Note we setup the ROS topic publishing and subscribing with these launch arguments.

```
    <!-- Launch the controller node -->
    <executable cmd="
        ros2 run ros_esc controller_node
        $(var entity_name)/filter_value_chatter
        /odom
        $(var entity_name)/timekeeper_chatter
        $(var entity_name)/control_value_chatter
        /cmd_vel
        $(var controller_config_filepath)
    "
    />
```

4.9 Data Collection Node

This node is used to subscribe to active ROS topics of interest, and collect the data published there to save to csv files for documentation. This node saves odometry data, sensor transform data, cost value data, filter output data, and controller output data with their respective timestamps. In addition to recording data to csv files, this node also creates a comments text file where all config files and objects used for this experiment are all recorded for future reference. All of these files will be saved in a folder marked "Test" and appended

with the date and time the experiment was conducted. The user can select the directory to store these "Test" folders in by editing the main gazebo launch file.

In addition to this, the data collection node is responsible for creating live plots of data that the user can reference during an ongoing simulation.

4.9.1 Launch Arguments

The data collection node expects a few input arguments from the user to launch correctly. These are given in the ordered list below.

1. The user must name the absolute filepath to a directory where the test folders will be saved to.
2. The user must name a rotate sensor frame configuration file that is used in the experiment, the config and object used will be recorded.
3. The user must name a transform configuration file that is used in the experiment, the config and object used will be recorded.
4. The user must name a cost function configuration file that is used in the experiment, the config and object used will be recorded.
5. The user must name a filter configuration file that is used in the experiment, the config used will be recorded.
6. The user must name a controller configuration file that is used in the experiment, the config and object used will be recorded.
7. The user must name an input timekeeper topic, this is used to collect Timekeeper messages containing timekeeping information to reference.
8. The user must name an input odometry topic, this is used to collect Odometry messages containing vehicle state data which will be saved to a csv file.
9. The user must name an input sensor transform topic, this is used to collect StampedTransformMultiArray messages containing information describing the position and orientation of sensors on the vehicle, this will be saved to a csv file.
10. The user must name an input cost topic, this is used to collect StampedFloat64MultiArray messages containing cost value data, this will be saved to a csv file.
11. The user must name an input filter topic, this is used to collect StampedFloat64MultiArray messages containing filter value data, this will be saved to a csv file.
12. The user must name an input control topic, this is used to collect StampedFloat64MultiArray messages containing control value data, this will be saved to a csv file.
13. The user can select a live plotting mode, either "None", "2D", or "3D".

These arguments are then used to form the launch command for the data collection node seen in 4.9.2.

4.9.2 Adding Node to Launch File

We now add the following items to the main Gazebo launch file. First, we initialize defaults for the variables necessary to launch this node

```
1   <!-- Set the directory to save simulation test results to -->
2   <arg name="data_collection_filepath" default="~/Documents/Gazebo_Simulations"/>
```

We then create a command to launch the rotate sensor frame node with the arguments given in 4.9.1. Note we setup the ROS topic publishing and subscribing with these launch arguments.

```
<!-- Launch the data collection node -->
<executable cmd="
    ros2 run ros_esc data_collection_node
    $(var data_collection_filepath)
    $(var rotate_frame_config_filepath)
    $(var sensor_transform_config_filepath)
    $(var cost_function_config_filepath)
    $(var filter_config_filepath)
    $(var controller_config_filepath)
    $(var entity_name)/timekeeper_chatter
    /odom
    $(var entity_name)/sensor_transform_chatter
    $(var entity_name)/cost_value_chatter
    $(var entity_name)/filter_value_chatter
    $(var entity_name)/control_value_chatter
    $(var live_plot_mode)
"
/>
```

V. Gazebo Simulations

With a completed launch file with all of the ROS ESC nodes, we can begin a simulation with all of those example config files and objects using the following commands.

```
user@machine:~$ cd ~/ros2_ws

user@machine:~$ colcon build --packages select ros_esc ros_esc_interfaces

                turtlebot3_rotating_sensor

user@machine:~$ source install/setup.bash

user@machine:~$ ros2 launch turtlebot3_rotating_sensor gazebo.launch.xml
```

The example config files and objects presented in the ROS ESC section correspond to an extremum seeker implementing the RMSpropESC method with a continuously rotating sensor frame. The objective function used is quadratic, with a minimum set at the coordinate location (3, 3). The seeker will find this extremum in the environment and, given enough time, begin to orbit around this point indicating convergence. When the Gazebo simulation launches the data collection node is responsible for creating a separate window to show live plots of the data being collected. This is useful testing the performance of new ESC methods since it allows one to visualize the data being transmitted between the ROS nodes.

If the user would like to speed up the simulation, they can do so by referencing figure 18, and by adjusting the real time factor accordingly. Please be advised that by adjusting the real time factor, you are also adjusting the publishing rate of all the ROS nodes. You can view the publishing rate on a particular topic of interest with the following command.

```
user@machine:~$ ros2 topic hz {topic_of_interest}
```

When adjusting the real time factor, you would want the publishing rate to scale by the same amount in order to get decent simulation results. The publishing rate starts to break down the faster you want the simulation to run however, so I recommend running simulations only up to 3 or 4 times speed.

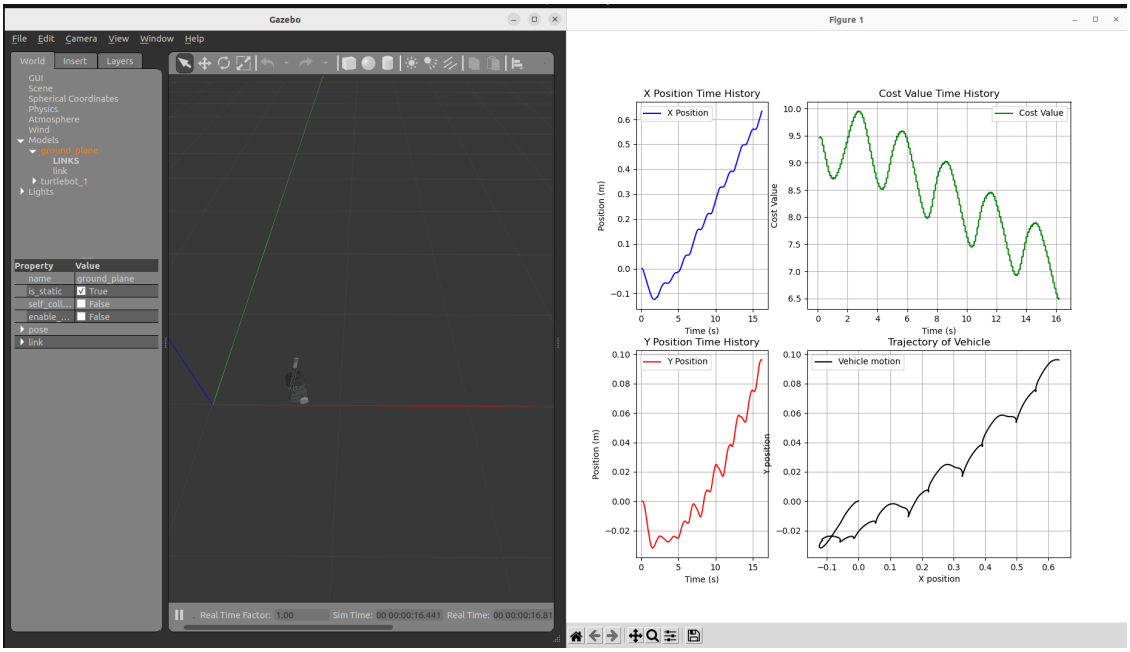


Figure 17: This shows what an active Gazebo simulation may look like with the live plots in a separate window.

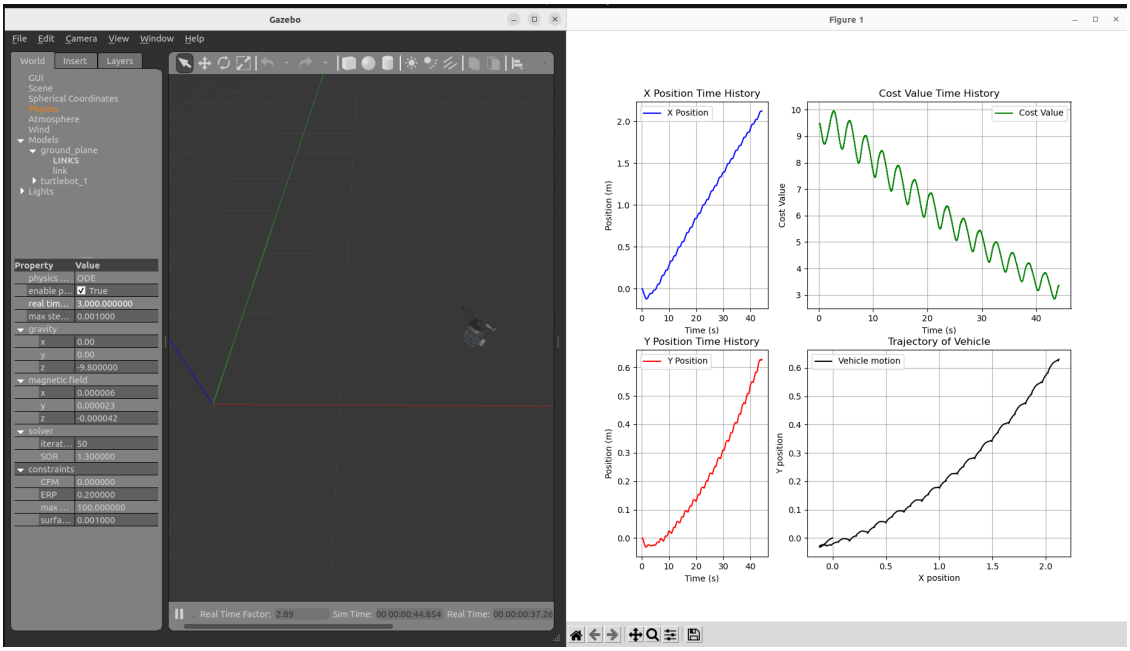
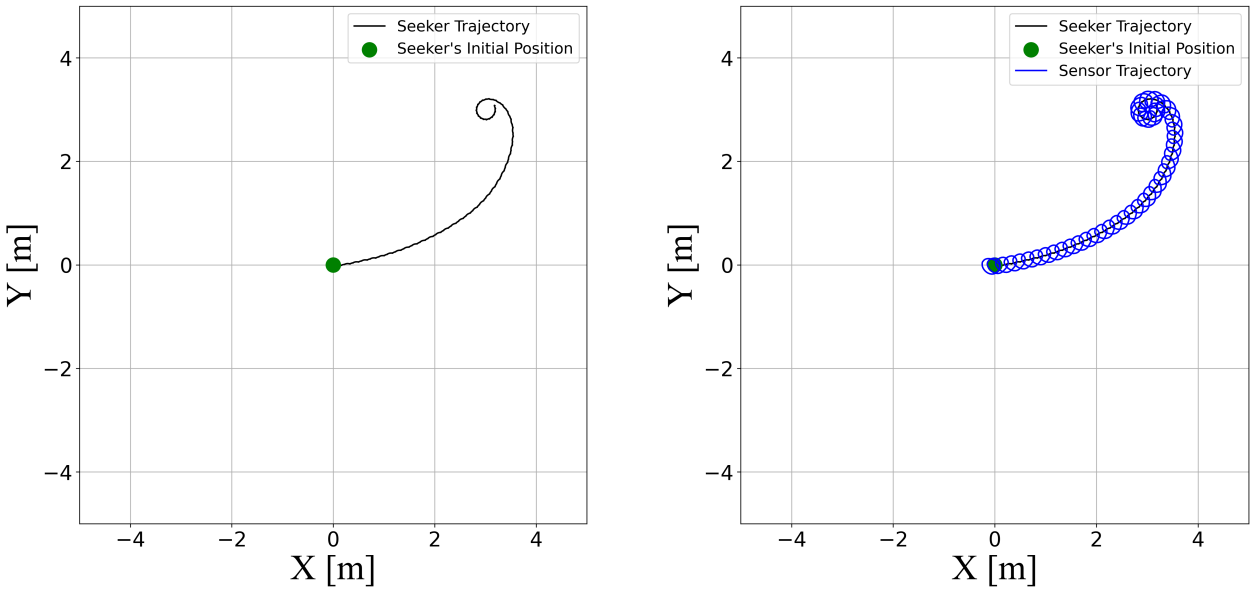


Figure 18: The user can adjust the time factor for a Gazebo simulation under the Physics, Real Time Factor property. Note x1 speed is a value of 1000, x2 speed is a value of 2000 etc.

Once the user is happy with their simulation results, they can close the simulation with ctrl+c and view all the data collected in the Test folder. In this folder they should find csv files with all the data collected by the data collection node, as well as a comments file describing everything used to conduct this simulation. To plot results from an experiment, the user can use one of the generic plotting scripts found in the ROS ESC package at the following location.

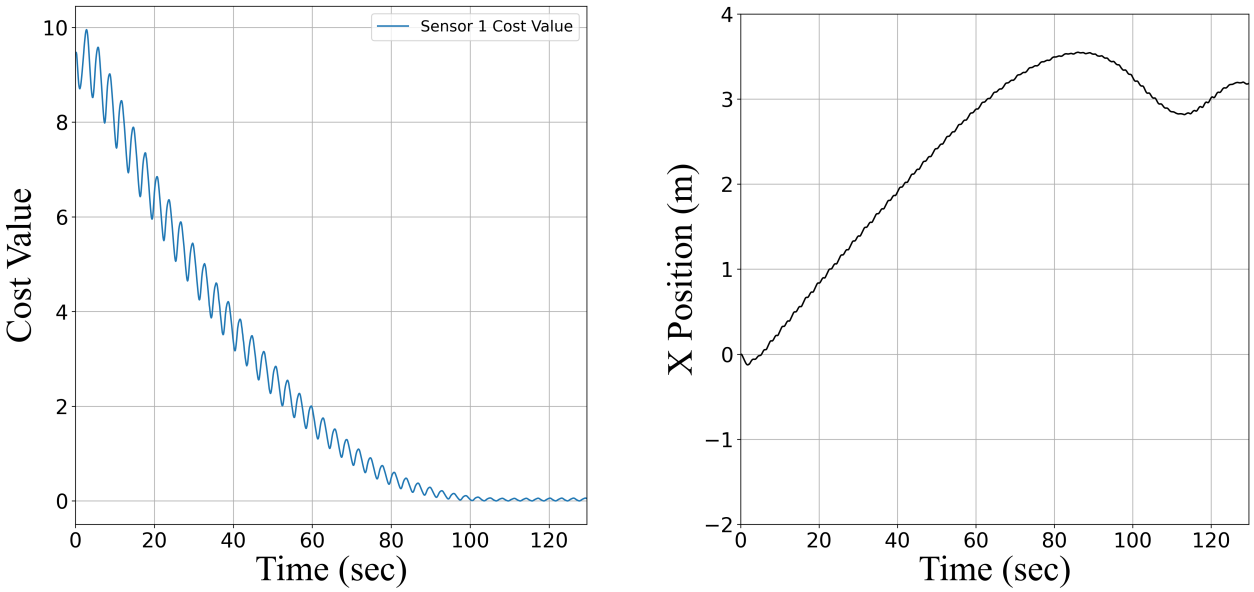
```
~/ros2_ws/src/ros_esc/ros_esc/data_collection_node/plotting_scripts
```

Figures 19 and 20 show some of the plots that can be generated by a plotting script written specifically for the Turtlebot3 vehicle.



(a) (b)

Figure 19: Example plots from the Gazebo simulation showing (a) the path the seeker took to get to the source, and (b) the same path with the sensor's position overlayed on top.



(a) (b)

Figure 20: Example plots from the Gazebo simulation showing (a) time history of the cost value, and (b) the time history of the vehicle's x position.

VI. Real Life Experiments

The ROS ESC package and ROS ESC Interfaces packages can be implemented on real life robotic vehicles to conduct extremum seeking experiments. The following section provides an example of implementing the ROS ESC functionality on a Turtlebot3 vehicle. Note that the Turtlebot3 Vehicle Nodes package seen in section 2.3.4 was created as an additional supplement to the ROS ESC package to accommodate for features that needed to be translated to real life hardware. Note in this section I assume you are using a Turtlebot3 vehicle that has already been setup with ROS. This section only describes how to implement these new packages on top of what is currently in place for ROS activities.

6.1 Implement ROS ESC on a Vehicle

To implement the ROS ESC and ROS ESC Interfaces package on a vehicle, one should first create a ROS workspace directory, and run through the steps shown in section 2 as needed. Note that vehicles may need additional software to function that is not mentioned here. Once the required packages are installed into the ROS workspace, the user should build the workspace with the following commands.

```
user@machine:~$ cd ~/ros2_ws

user@machine:~$ colcon build
```

They can also build select packages using the following commands.

```
user@machine:~$ cd ~/ros2_ws

user@machine:~$ colcon build --packages-select ros_esc ros_esc_interfaces

                turtlebot3_vehicle_nodes
```

Note that the Turtlebot3 rotating sensor package does not need to be installed on the vehicle as it is a simulation only package.

6.2 Creating New Nodes

One may need to develop new ROS nodes to interface with existing hardware, sensors, systems etc. For the Turtlebot3 vehicle, this lead to the creation of the supplemental package called Turtlebot3 Vehicle Nodes. The Encoder node, Rotate Sensor Frame node, Sensor Pose node, and Cost Function node are designed primarily for Gazebo experimentation, they needed to be replicated for the system on the real Turtlebot3 vehicle.

6.2.1 Encoder Node

The Turtlebot3 vehicle uses a Parallax Feedback 360 servo motor to drive the rotation of the rotating sensor frame. The product page for this part is found at this link. The servo has a built in absolute rotary encoder, to communicate with this sensor we design a new encoder node specific to the Turtlebot3 vehicle. This node performs the exact same functions as the encoder node in the ROS ESC package, but instead of getting the angular position from some ROS topic, now it gets it from sensor feedback.

The pin description for the feedback signal is given in figure 21. The equation used to calculate the angular position of the servo motor is pulled from the product documentation and is shown in figure 22.

This encoder node has similar launch arguments as the one in the ROS ESC package.

1. The user must name an input timekeeper topic, this is used to collect Timekeeper messages which contain timekeeping information to reference.

2. The user must name an output topic to publish StampedFloat64MultiArray messages to, these messages contain angular position data.

3. Using the "–publish_rate" argument, the user can set the publishing rate of this encoder node, a publishing rate of at least 20 Hz is recommended.

Pin Descriptions

Pin	Name	Description	Min	Typical	Max	Units
Yellow	Feedback	Servo output, PWM, 910 Hz, 2.7–97.1% duty cycle		3.3		V
White	Control	Servo input, PWM, 50 Hz, 1280–1720 µs	3.3	3.3	5.0	V
Red	Vservo	Servo power supply	5*	6	8.4	V
Black	Vss	Ground, common ground with microcontroller		0		v

*5 VDC is absolute minimum required for no-load angular position control. 5.8 to 8 VDC is recommended for continuous rotation speed control.

Figure 21: Parallax servo motor pin descriptions

For example, a rolling robot application may use 64 units, or "ticks" per full circle, mimicking the behavior or a 32-spoke wheel with an optical 1-bit binary encoder. A fixed-position application may use 360 units, to move and hold the servo to a certain angle.

The following formula can be used to correlate the feedback signal's duty cycle to angular position in your chosen units per full circle.

Duty Cycle = 100% x (tHigh / tCycle). Duty Cycle Min = 2.9%. Duty Cycle Max= 97.1%.

$$\textit{Angular position in units full circle} = \frac{\textit{(Duty Cycle - Duty Cycle Min) x units full circle}}{\textit{Duty Cycle Max - Duty Cycle Min + 1}}$$

Figure 22: The equation used to calculate the angular position with the Parallax servo motor.

6.2.2 Rotate Frame Node

This node performs the exact same functions as the rotate frame node in the ROS ESC package, but instead of using Gazebo ROS2 Control to send and enact velocity commands, this node must convert the velocity command to a PWM signal to send to the Parallax servo motor. This node uses the same configuration file system as the one in the ROS ESC package.

The velocity command to PWM signal conversion is based on the experimental data seen in figure 23. Note linear fits were computed for clockwise and counterclockwise rotation, these equations convert a desired velocity to a PWM value.

This rotate frame node has similar launch arguments as the one in the ROS ESC package

1. The user must name an input encoder topic, this is used to collect StampedFloat64MultiArray messages containing angular position data.

2. The user must name an output topic to publish Timekeeper messages to, note the instant the sensor frame starts to rotate is considered the start time of the experiment, this is communicated to other ROS nodes via this message. Note all ROS nodes will now be using real time.

3. The user must name a rotate sensor frame configuration file to use, this gives the rotate sensor frame node a configured object to use in an experiment.

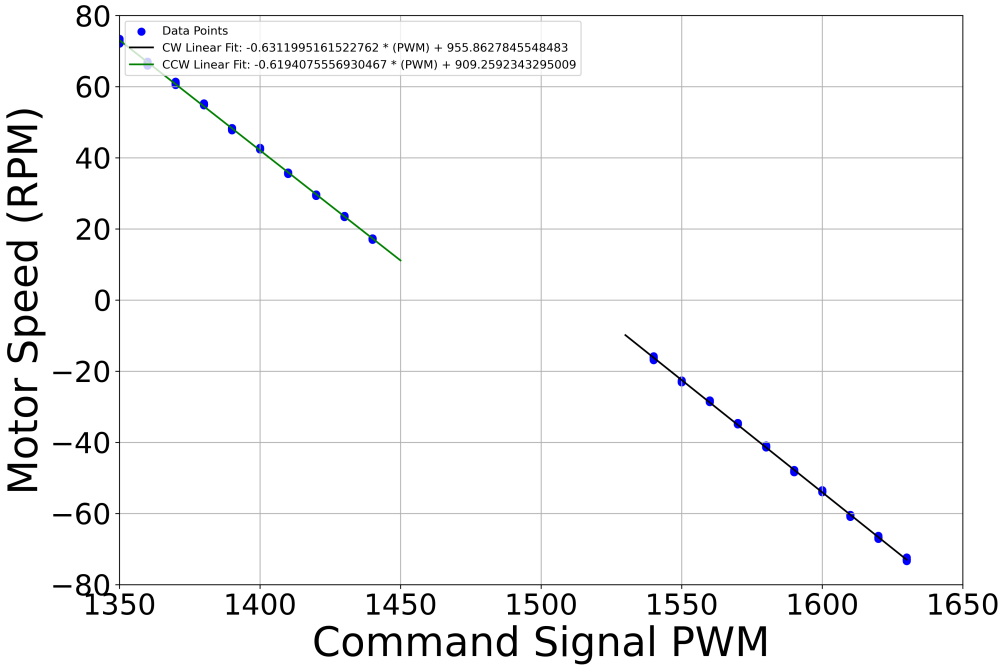


Figure 23: A rotation speed test for the Parallax servo motor

6.2.3 Microphone Node

This node is used to read and relay microphone data sent via a serial connection by another board used for sensor data processing. Instead calculating a cost value based off of some preset objective function in a simulation, in a real life experiment the cost value comes from a sensor reading. The microphone node is used for experiments implementing acoustic based extremum seeking schemes.

There are some additional scripts that must be installed on a separate board used for sensor data processing. For the microphone sensor, this board is a separate Raspberry Pi with a MCC172 Daqhat board used to collect and store samples. All files from the following location should be installed on the separate RPi board for acoustic signal data analysis. Additional setup instructions from the README file in this directory must be followed to complete the installation.

```
~/ros2_ws/src/turtlebot3_vehicle_nodes/microphone_scripts
```

We are currently testing multiple ways of developing cost function values with microphone data, the data analysis function config files can be found at the following location.

```
~/ros2_ws/src/turtlebot3_vehicle_nodes/microphone_scripts/data_analysis_function_config_files
```

To start collecting acoustic data, the mic_scan_script must be run as an executable that can be configured with additional arguments. The microphone scan executable has the following launch arguments.

1. The user must name a data analysis config file to use to process acoustic data.
2. Using the "–omega_l" argument, one can set the low pass filter parameter used to filter cost value data. All cost value data is put through a low pass filter to help reduce the noise.
3. Using the "–publish_hz" argument, one can adjust the publishing speed of the cost value data as it gets sent via serial connection to the RPi running the ROS nodes. Note that all data collection and filtering still occurs at the fastest rate possible, separate from the publishing rate of this executable.
4. Using the "–num_samples" argument, one can adjust the total number of samples used to compute an acoustic cost value using the requested data analysis function in the config file.

This microphone node has the following launch arguments seen below.

1. The user must name an input timekeeper topic, this is used to collect Timekeeper messages with timekeeping information to reference.

2. The user must name an output topic to publish StampedFloat64MultiArray messages to, these messages contain acoustic based cost value data.

6.2.4 Photoresistor Node

This node is used to read and relay photoresistor data sent via a serial connection by another board used for sensor data processing. The photoresistor node is used for experiments implementing light based extremum seeking schemes.

There are some additional scripts that must be installed on a separate board used for sensor data processing. For the photoresistor sensor, this board is an Arduino which is wired to a circuit containing a photoresistor sensor. A working version of Arduino IDE can be used to upload the following script to the Arduino board.

```
~/ros2_ws/src/turtlebot3_vehicle_nodes/photoresistor_scripts/photoresistor.ino
```

This script will run continuously whenever the Arduino board is powered on. It does not need to be launched as an executable.

This photoresistor node has the following launch arguments seen below.

1. The user must name an input timekeeper topic, this is used to collect Timekeeper messages with timekeeping information to reference.

2. The user must name an output topic to publish StampedFloat64MultiArray messages to, these messages contain light based cost value data.

3. The user must select if they desire 'Voltage' or 'Resistance' readings from the sensor.

6.2.5 Vicon Communication Node

The DSIM Lab has access to a Vicon motion capture system that may be used to track the vehicle's position during an experiment. This is an optional functionality, real life extremum seeking experiments can be conducted without Vicon based position measurements. This node merely relays odometry data from the Vicon system which conveys the Turtlebot3 vehicle's position and orientation in the lab environment. This data is recorded by the data collection node and saved locally after an experiment is completed. For more information on the Vicon system, please see the following guide on the DSIM google drive.

This Vicon communication node has the following launch arguments seen below.

1. The user must name an input timekeeper topic, this is used to collect Timekeeper messages with timekeeping information to reference.

2. The user must name an output topic to publish StampedFloat64MultiArray messages to, these messages contain odometry data from the vicon system.

6.2.6 Data Collection Node

The data collection node in the Turtlebot3 Vehicle Nodes package is a slightly modified version of the one seen in the ROS ESC package. This node takes similar launch arguments an documents all experimental data into test folders stored locally on the vehicle. Note that test folders should be periodically cleaned out of the vehicle's file system to free up space.

6.3 Creating a Launch File

For the Turtlebot3 vehicle nodes package, we develop a new launch file to use the new nodes we have discussed, and some of the other nodes from the ROS ESC package. We can use the same config files from simulation on the real life robot if we specify them in the way we did before. First, for the Turtlebot3 vehicle, we need an additional launch script to perform the vehicle bringup. This script gets the robot prepared to run ROS actions, and prepares the motors on the bottom of the vehicle to be ready to drive around. This script can be found at the following location.

```
~/ros2_ws/src/turtlebot3_vehicle_nodes/launch/vehicle_bringup.launch.py
```

We then initialize the ROS node communication and create launch commands in a manner similar to before with the Gazebo simulation. At the following location, one can find a launch file which launches most of the individual nodes we have discussed in the previous sections. These nodes will be launched for an extremum seeking experiment regardless of the choice of sensor used in the test. The only nodes that do not appear in this launch file are those related to initializing and relaying sensor cost value data.

```
~/ros2_ws/src/turtlebot3_vehicle_nodes/launch/nodes.launch.xml
```

The following are two different examples of launch files for extremum seeking experiments with two different sensors. These launch files will utilize the nodes launch file above, but they also initialize the unique sensor node needed for their respective experiments.

```
~/ros2_ws/src/turtlebot3_vehicle_nodes/launch/acoustic_esc_experiment.launch.xml
```

```
~/ros2_ws/src/turtlebot3_vehicle_nodes/launch/light_esc_experiment.launch.xml
```

6.4 Running an Experiment

On startup, there are several things that need to be taken care of before any experiments are run with the Turtlebot3 vehicle. First we must activate the pigpio daemon to be able to use the servo motor via the GPIO pins. Next we set the date and time on the RPi to the correct values so that "Test" folders are generated with the appropriate timestamp. To activate the servo motor, use the following command and then use the password "turtlebot" to activate the pigpio daemon.

```
user@machine:~$ sudo pigpiod
```

To set the date and time on the RPi, use the following command. This must be done every time the Turtlebot is turned on since the date resets.

```
user@machine:~$ sudo date -s "{Month} {Day} {Hour}:{Min} {Year}"
```

If the user is using the microphone sensor in an experiment, refer to section 6.2.3 for instructions on how to launch the mic scan executable on the separate data collection board. Finally, the user can begin an experiment with the following commands

```
user@machine:~$ cd ~/ros2_ws

user@machine:~$ colcon build --packages-select ros_esc turtlebot3_vehicle_nodes

user@machine:~$ source install/setup.bash

user@machine:~$ ros2 launch turtlebot3_vehicle_nodes vehicle.launch.xml
```

VII. Future Additions to ROS ESC

If the user requires something new from the ROS ESC package they need to create a new object to realize that idea and make a config file to configure it. The general strategy for adding future additions is as follows.

1. Design an object for a ROS ESC node to use, note it must abide by the expected inputs and outputs for the respective node. However how one gets from the inputs to the outputs is entirely up to the user to design.
2. Create a config file to configure that object. This should include any adjustable parameters that a user might want to play with during experimentation.
3. Test the new object and config file in simulations to verify it works with existing software.
4. Submit a merge request with new material to be evaluated by a DSIM Lab Gitlab maintainer.

Users to the ROS ESC and associated packages should create a new branch off of the main using the following command as an example. Within this new branch they can add new config files or objects, and experiment with them as they please.

```
user@machine:~$ cd ~/ros2_ws/src/ros_esc

user@machine:~$ git branch {new_branch_name}
```

If users would like to add their new config files or objects to the package, they should contact a DSIM Lab Gitlab maintainer so they can evaluate new additions and merge requests to this package.